

SEDAS

Social Engineering Defence and Awareness System: An Integrated Machine Learning Platform for Phishing Detection and User Awareness

MIS5320 Applied IS Project — Part B · Assessment 3: Final Project Report

Team	Ashraful Islam (240135); Huzaifa Iqbal (240220); Sonu (250001); Dominic Kipchirchir (240401)
Supervisor	Dr Chandana Withana
Institution	Apex Australia Higher Education
Base paper [41]	Al-Subaiey et al. (2024), Computers and Electrical Engineering, 120:109625 (Q1)
Repository	github.com/ailinkon/MIS5320-SEDAS-CyberProjectB
System version	0.1.0

Table of Contents

Abstract	4
Executive Summary	5
1. Introduction	6
1.1 Background and Motivation	6
1.2 Problem Statement	6
1.3 Requirements and Scope	6
1.4 Contributions.....	7
1.5 Report Structure	8
2. Literature Review	9
2.1 Phishing and Social Engineering: Attack Landscape.....	9
2.2 Machine Learning Approaches to Phishing Detection	9
2.3 Deep Learning and Transformer-Based Detection	10
2.4 Explainable AI in Security Applications	10
2.5 Security Awareness Training and Human Factors	11
2.6 Trust and Integrity Verification Mechanisms	12
2.7 Research Gap	12
3. Methodology	14
3.1 Research Design.....	14
3.2 System Architecture	14
3.3 Dataset.....	16
3.4 Data Model	16
3.5 Preprocessing and Feature Extraction.....	17
3.6 Algorithms Investigated	18
3.6.1 Support Vector Classifier (LinearSVC).....	18
3.6.2 Random Forest	18
3.6.3 Multinomial Naive Bayes	18
3.6.4 Logistic Regression (extension).....	19
3.6.5 XGBoost (extension).....	19
3.7 Extension Experiments	19
3.8 Transformer-Based Approach	20
3.9 Refinements from the Part A Design	20
3.10 Evaluation Protocol	21
4. System Implementation and Deployment	23
4.1 Detection Service.....	23
4.2 URL Verification Service	23
4.3 Awareness Training Service	24
4.4 Governance and Interface Layer.....	25
4.5 Technology Stack	25

4.6 Security Controls	26
4.6.1. Methodology and Scope	26
4.6.2. Findings by Category	27
4.6.3. Summary of Findings	30
4.6.4. Prioritised Remediation Roadmap	30
4.6.5. Limitations of This Assessment	31
4.7 Testing and Quality Assurance	31
4.7.1 Testing strategy	31
4.7.2 Security testing	32
4.7.3 Defect identified through testing	32
5. Results and Discussion	33
5.1 Benchmark Replication Results	33
5.2 Metric-Level Analysis	34
5.3 Operational Metrics Not Reported in the Base Paper	35
5.4 Extension Experiments and the Performance Ceiling	36
5.5 Transformer-Based Comparison (Preliminary)	38
5.6 Precision-Recall Trade-Off	40
5.7 Significance of Findings	41
5.8 Limitations	42
5.9 Outcomes Against Objectives	42
5.10 Design Trade-Offs	43
6. Conclusion and Future Work	45
6.1 Conclusion	45
6.2 Future Work	45
References	47
Base paper	47
Additional references added in Part B	47
Appendices	48
Appendix A: Complete Results Data	48

Abstract

Phishing Prevention: Social Engineering Defences and Awareness System (SEDAS) — An Integrated Machine Learning Platform for Phishing Detection and Awareness

Beyond technical sections, phishing is often a social engineering attack that can be carried out as long as the user's judgment is tricked as well. The challenge extends especially to the higher education context where large and often volatile user populations, decentralised digital services, important identity, financial and research information forms a wide 'attack surface'. The Social Engineering Defence and Awareness System (SEDAS) is a comprehensive defensive solution that does more than simply classify email messages as phishing and non-phishing, and leverages explainable phishing detection, URL verification, tamper evident threat record, adaptive security-awareness training, and governance oriented data capture.

A two-stage methodology, independent, benchmark replication and systematic extension, were implemented. It reproduced the benchmark with the same public dataset of 82,486 emails, including phishing messages and legitimate emails. The 80/20 stratified split was used in classical experiment with a fixed random seed to promote experiment repeatability. A total of 6 classification approaches were explored: LinearSVC, Random Forest, Multinomial Naive Bayes, Logistic Regression, XGBoost and a simple DistilBERT transformer method.

The experimental results were able to reproduce the benchmark results successfully. LinearSVC was found to be the best classical model with an F1 score of 99.16% showing near matching with the published baseline. Random Forest- 98.71%, Logistic Regression- 98.65%, XGBoost- 97.56% and Multinomial Naive Bayes- 97.47%. LinearSVC also showed well balanced operational behaviour with high precision and recall.

Four controlled optimisation experiments were performed: One with engineered features, one with voting ensembles, one with weighted voting, and one with regularisation tuning. All except for none achieved no significant improvement over the baseline, thus showing that the TF-IDF/classical configuration being tested had run its course in terms of achieving higher performance on this dataset.

A preliminary test on DistilBERT, a transformer model experiment which tested contextual modelling under the CPU-only constraint. With the reduced stratified data set, DistilBERT's F1 score was 98.00%. The transformer experiment and the classical model have significantly different amounts of data; so the result was not analysed as directly comparable, but indicative. It was technically feasible and viable for future evaluation in full-scale transformer.

In addition to classification performance, SEDAS adds an integrated architecture comprising a browser interface, of course FastAPI services and separate data stores. The system offers phishing categorisation, explainable outputs, URL checks in a SHA-256 hash-chained registry, adaptive learning in awareness and governance-oriented records.

The impact of SEDAS is not about a slight gain in accuracy, but a reproducible systematic validation of a robust benchmark, evidence of limit of optimisation and security for operations and a full continuum from machine learning detection to human awareness and institutional governance. Future plans involve full scale evaluation of transformer, enhanced authentication and access control, institutional integration, and ethical behaviour studies.

Executive Summary

Phishing remains a persistent form of social engineering because technical filtering alone cannot remove the human judgement on which deceptive messages ultimately depend. In higher-education environments, large and changing user populations, decentralised services and valuable identity, financial and research data create an attractive attack surface. Detection alone is therefore insufficient if users receive no explanation, unsafe links are not remembered, and security outcomes do not feed back into training or governance. This project developed SEDAS, an integrated defence and awareness platform combining phishing-email classification, explainable outputs, tamper-evident URL verification, adaptive awareness training and governance-oriented records. The study first reproduced the benchmark of Al-Subaiey et al. [41] on the identical public corpus of 82,486 emails, then extended the evaluation to six methods and four optimisation strategies. LinearSVC achieved an F1 score of 99.16%, closely confirming the published 99.0% benchmark. Random Forest achieved 98.71%, Logistic Regression 98.65%, XGBoost 97.56% and Multinomial Naive Bayes 97.47%. A preliminary DistilBERT experiment achieved 98.00% F1 on a substantially smaller CPU-constrained subsample and is therefore treated as indicative rather than directly comparable. Four further attempts to exceed the SVC benchmark—engineered features, equal voting, weighted voting and regularisation tuning—converged within a narrow performance band, identifying a practical ceiling for the evaluated TF-IDF/classical configuration. Beyond headline F1, SEDAS reports false-positive rates, absolute error counts, confusion matrices and measured latency, exposing operational differences hidden by aggregate scores. Its contribution is therefore not a marginal numerical improvement, but a reproducible benchmark replication, an empirically supported ceiling finding, richer operational evaluation, and integration of validated detection with URL integrity controls, adaptive human-layer defence and governance evidence.

Keywords: phishing detection, social engineering, machine learning, TF-IDF, security awareness training, explainable AI, tamper-evident logging

1. Introduction

1.1 Background and Motivation

Phishing is one of the most persistent social-engineering threats because it does not need to defeat an organisation's strongest technical controls; it only needs to persuade a user to trust a deceptive message, link or request. The problem is particularly relevant to higher education. Universities support large and frequently changing populations of students, staff, contractors and visitors, while also holding identity information, financial records, research data and privileged access to institutional systems. Their decentralised structures and diverse digital services create many plausible identities and workflows that attackers can imitate. The resulting environment makes urgent requests about passwords, enrolment, payroll, shared documents or account verification appear credible enough to trigger action before careful checking occurs.

Detection technology has improved substantially, and recent studies report very high classification performance for both classical and transformer-based approaches [41], [49]-[51]. However, social engineering remains a human-centred problem. Awareness research shows that the design, timing and feedback characteristics of training influence whether users transfer knowledge into safer behaviour [45], while broader reviews indicate that adaptive and reinforced interventions are more effective than isolated awareness exposure [56]. At the same time, AI-generated phishing is increasing the linguistic quality and variation of deceptive content, reducing reliance on obvious grammar or spelling mistakes as warning signs [43], [57]. These developments create a gap between accurate technical detection and resilient user behaviour. SEDAS is motivated by that gap: it treats classification as one defensive layer and extends it with URL verification, interpretable feedback, adaptive training and governance evidence so that a detected threat can also contribute to future prevention.

1.2 Problem Statement

This project addresses two connected problems. The first is a field-level reproducibility problem. Phishing-detection studies often use datasets, preprocessing pipelines or configurations that are not fully available to other researchers, making it difficult to determine whether reported gains result from a genuinely better method or from differences in data preparation and evaluation. The base study [41] explicitly responds to this issue by publishing results on a large public corpus. SEDAS therefore begins with an independent reproduction on the identical 82,486-email dataset and fixes the train/test procedure and random seed so that its own comparisons can be independently checked.

The second problem concerns the scope of phishing defence. Most detection research evaluates a classifier as an isolated technical artefact: an email is labelled, metrics are reported, and the evaluation ends. Social engineering, however, succeeds through user judgement and organisational response as well as message content. A highly accurate classifier does not itself teach a user why a message was suspicious, prevent a malicious URL from being trusted later, adapt awareness content to demonstrated risk, or provide governance records showing recurring patterns. Consequently, technical detection and human-layer risk management are frequently disconnected. SEDAS addresses this specific gap by integrating explainable classification with tamper-evident URL verification, adaptive awareness training and auditable records. The research question is therefore not only how accurately phishing can be classified, but how validated detection can be embedded within a broader defensive workflow while remaining reproducible and operationally measurable.

1.3 Requirements and Scope

ID	Functional requirement	Status
FR-01	Classify submitted email text and return a phishing risk score	Delivered
FR-02	Provide human-readable explanation of contributing indicators	Delivered
FR-03	Verify URLs against heuristic analysis and a trust registry	Delivered
FR-04	Record verifications in a tamper-evident append-only log	Delivered

ID	Functional requirement	Status
FR-05	Deliver persona-based micro-lessons adaptively	Delivered
FR-06	Track per-user training progress and risk score	Delivered
FR-07	Present role-based governance analytics	Partial — data captured, dashboard deferred
FR-08	Enforce role-based access control	Deferred — see Section 4.6.1
FR-09	Expose all functions via documented RESTful APIs	Delivered
FR-10	Integrate with institutional LMS, SIEM and SSO	Deferred — refinement R-03

Table 3: Functional requirements and delivery status.

ID	Non-functional requirement	Target	Achieved
NFR-01	Detection quality (F1)	≥ 0.95	0.9917
NFR-02	URL verification latency	$< 2,000$ ms	≈ 1.4 -9.3 ms
NFR-03	Classification latency	$< 2,000$ ms	≈ 7 -11 ms
NFR-04	Availability	Best effort (dev)	Not measured
NFR-05	Security of the artefact	OWASP-informed controls	Partial — §4.6
NFR-06	Privacy	Public/synthetic data only	Achieved
NFR-08	Auditability	Timestamped records	Achieved

Table 4: Non-functional requirements against measured outcomes.

Tables 3 and 4 define the implementation boundary used for the Part B minimum viable product. In scope are four connected capabilities: explainable email classification, URL verification with a tamper-evident registry, persona-based adaptive awareness training, and governance-oriented data capture exposed through documented REST APIs. These components are sufficient to demonstrate the complete defensive workflow while keeping the implementation feasible within the unit timeframe. Production authentication and role-based access control, live LMS/SIEM/SSO integration, a distributed blockchain ledger and production-scale availability engineering are explicitly outside the MVP. Those items depend on institutional identity infrastructure, external access, deployment resources or governance approvals that were not available to a student project. The scope was not reduced informally during development; it was formally re-baselined through the eight-item refinement register discussed in Section 3.9, preserving the original design intent while removing enterprise infrastructure that could not be meaningfully evaluated. Within this scope the study asks whether the published benchmark can be reproduced, whether classical optimisation can materially improve it, whether contextual transformer modelling is viable, and whether detection can be integrated with human-layer and governance controls in one system.

1.4 Contributions

SEDAS makes five contributions that extend beyond reproducing a published classification score. First, it provides an independently re-runnable replication of the Q1 benchmark in [41] on the identical public dataset. The corpus composition, 80/20 stratified split, preprocessing configuration, random seed and evaluation metrics are disclosed, allowing a third party to regenerate the experiment rather than relying on an opaque summary result.

Second, the study evaluates six methods within one evidence framework. LinearSVC, Random Forest and Multinomial Naive Bayes reproduce the base paper's classical methods; Logistic Regression and XGBoost extend the classical comparison; and DistilBERT tests a fundamentally different contextual representation at preliminary scale. The reduced DistilBERT experiment is explicitly separated from the full-data classical comparison to avoid unsupported equivalence.

Third, four systematic optimisation strategies were tested after replication: engineered features, equal-weight

voting, SVC-weighted voting and regularisation tuning. None produced a meaningful improvement, with results converging around 99.16-99.17% F1. This establishes an empirical performance-ceiling finding for the evaluated TF-IDF/classical approach and shows that further small adjustments are unlikely to yield substantive benefit.

Fourth, SEDAS reports operational metrics not provided by [41], including false-positive rates, absolute false-positive and false-negative counts, complete confusion matrices and measured request latency. These reveal deployment behaviour that headline F1 alone conceals.

Fifth, the validated classifier is placed inside an integrated architecture combining explainable detection, a SHA-256 hash-chained URL registry, adaptive awareness training and governance records. The contribution is therefore both empirical and architectural: reproducible evidence about method limits, plus a working pathway from technical detection to human-layer defence and institutional oversight.

1.5 Report Structure

The remainder of this report follows the sequence of the benchmark paper [41] while incorporating the additional artefact requirements of the assessment. Section 2 reviews phishing detection, transformers, explainable AI, awareness training and integrity mechanisms. Section 3 describes the research design, architecture, dataset, preprocessing, algorithms, extensions and evaluation protocol. Section 4 presents the implemented services, security controls and testing. Section 5 reports and critically discusses model performance, operational errors, optimisation experiments and the preliminary transformer comparison. Section 6 concludes the study, answers the guiding questions and identifies future development and evaluation priorities.

2. Literature Review

2.1 Phishing and Social Engineering: Attack Landscape

Phishing has evolved from easily recognised bulk spam into a broader family of social-engineering attacks that exploit identity, context and urgency. Conventional email phishing remains important, but organisations must also contend with spear-phishing targeted at particular individuals, business-email-compromise scenarios that impersonate trusted decision makers, credential-harvesting links, fraudulent account-verification requests and messages that redirect users to lookalike websites. These techniques are effective because they manipulate routine organisational behaviour rather than relying solely on technical exploitation. In higher education, attackers can imitate lecturers, administrators, cloud platforms, payment services or collaboration tools, making legitimate operational language itself part of the attack surface.

Recent literature indicates that generative AI is changing this landscape further. Opara et al. [43] examine the ability of conventional filters and stylometric methods to identify AI-generated phishing, highlighting the challenge created when attackers can cheaply produce fluent, personalised and grammatically consistent messages. Reference [57] similarly examines machine-learning and watermarking approaches for identifying AI-generated phishing content. These developments weaken older user heuristics that depend on poor spelling or awkward phrasing. They also increase the potential volume and variation of attacks because content can be adapted quickly to different roles, institutions and current events. Consequently, phishing defence increasingly requires multiple complementary layers: content classification, URL inspection, interpretable feedback and sustained user awareness. This evolution provides the threat context for SEDAS, which is designed around the assumption that no single detector can address the complete social-engineering pathway.

2.2 Machine Learning Approaches to Phishing Detection

Classical machine learning remains a strong baseline for phishing email detection despite the rise of deep learning. The benchmark study underpinning this project, Al-Subaiey et al. [41], reported an F1 score of 0.99 using a Support Vector Machine on TF-IDF features across the largest available public dataset, outperforming Random Forest and Multinomial Naive Bayes on the same data. This finding is corroborated by a broader multi-model benchmark [49], which evaluated fourteen machine learning and deep learning models across ten datasets and found deep learning models consistently outperforming classical counterparts, though classical methods such as SVM remained competitive baselines. A related evaluation using SVM, Logistic Regression, Random Forest and Gradient Boosting on a Kaggle-sourced email dataset reported accuracies between 93% and 97%, with SVM again the strongest classical performer, reinforcing the consistency of SVM's advantage across independent studies and datasets.

Classical machine-learning techniques continue to provide a strong reference point for phishing-email detection because sparse text representations such as TF-IDF can encode highly discriminative vocabulary at low computational cost. In the benchmark that anchors this study, Al-Subaiey et al. [41] used TF-IDF with several supervised classifiers and reported Support Vector Machine as the strongest model, with an F1 score of approximately 0.99 on the merged public corpus. That result is consistent with the wider comparative literature, where SVM frequently remains competitive even as more computationally intensive deep-learning methods are introduced. The multi-dataset study in [49], for example, compares fourteen machine-learning and deep-learning approaches and finds that deep models often lead overall, while still confirming the value of classical baselines for robust benchmarking.

The Part A comparative analysis similarly showed a recurring pattern across logistic models, tree ensembles, Naive Bayes, SVM and hybrid neural approaches: performance depends not only on classifier family but

also on feature representation, corpus composition, false-positive tolerance and deployment constraints. SVMs are particularly well matched to high-dimensional sparse email vectors; Logistic Regression provides a fast interpretable linear alternative; Random Forest offers nonlinear ensemble behaviour; and Multinomial Naive Bayes provides an efficient probabilistic baseline. Recent studies [49], [51], [53] extend that comparison toward deep transformers and boosted trees. This literature therefore justifies SEDAS's staged methodology: first replicate the strongest established classical benchmark under controlled conditions, then test additional classical and transformer approaches without changing the underlying evaluation framework.

2.3 Deep Learning and Transformer-Based Detection

Transformer-based language models have increasingly been applied to phishing detection, motivated by their capacity to model word context and order, which bag-of-words representations such as TF-IDF cannot capture. A direct comparison of classical models against transformer models found the transformers achieved significantly higher performance, with RoBERTa reaching 99.43% accuracy [51]. A separate BERT-versus-RoBERTa study using a Nazario-and-Enron merged corpus — the same corpus family used in this project — found RoBERTa produced fewer false negatives than BERT, an important consideration given that false negatives represent undetected attacks [50]. However, not all findings unambiguously favour transformers: the large-scale benchmark [49] found transformer models outperformed classical approaches by an average margin of only 4.7%, suggesting the advantage, while real, is more modest than headline comparisons sometimes imply. Adaptive optimisation of hybrid deep learning architectures has also been explored [42]. These findings collectively motivate and contextualise the transformer comparison undertaken in Section 5.5.

Deep-learning and transformer models are increasingly used in phishing detection because they can represent contextual relationships that are lost when email text is reduced to independent weighted terms. TF-IDF records whether words or short n-grams are informative, but a transformer can model how a word's meaning changes according to surrounding tokens and sentence order. This distinction is relevant to social engineering, where the same vocabulary may be benign or malicious depending on how requests, urgency and identity claims are combined.

Recent evidence generally supports the potential of contextual models. The comparative study in [51] reports stronger performance for transformer architectures than for several traditional machine-learning baselines, with RoBERTa reaching 99.43% accuracy. Reference [50], using a Nazario-and-Enron corpus from the same dataset family as SEDAS, reports fewer false negatives for RoBERTa than BERT, which is operationally important because false negatives represent phishing that reaches users without warning. However, the advantage is not uniform across all studies. The broader evaluation in [49] reports an average transformer margin of about 4.7% over classical methods, suggesting that improvements can be smaller than headline single-dataset results imply. Hybrid deep architectures with adaptive optimisation have also been explored [42]. These findings directly motivate Section 5.5: SEDAS evaluates DistilBERT to test whether contextual representation is viable on the same task while explicitly acknowledging the CPU-driven subsampling constraint.

2.4 Explainable AI in Security Applications

Explainable AI has become a prominent theme in recent phishing detection research, addressing the trust and transparency limitations of black-box classifiers. Al-Subaiey et al. [41] integrated LIME to support user trust in individual predictions. This approach is extended in several recent studies: an XGBoost-and-SHAP framework for phishing website detection achieved 97.78% across accuracy, precision, recall and F1, using SHAP for both global and local interpretability [53]; a hybrid explainability framework combining SHAP, LIME, partial

dependence plots and permutation importance was applied to approximately 500,000 web pages from four independent sources to drive feature selection [54]; and a temporally dynamic feature-extraction framework combined LIME and SHAP with time-aware features [52]. Explainable AI has also been applied to web phishing classification in IoT and cloud-based cyber-physical contexts [44]. Beyond phishing specifically, a broader taxonomy situates these techniques within the wider challenge of building trustworthy security systems [48], while transformer-specific explainability has been demonstrated for adjacent text-classification tasks such as SMS spam detection [47] and for phishing email detection using large language models [46]. SEDAS's coefficient-weighted explainability approach, detailed in Section 3.6, follows this established pattern of pairing a classifier with an interpretable output layer, while adopting a lighter-weight technique appropriate to the linear models identified as strongest in this study.

Explainability is increasingly important in security classification because a prediction is more useful when users and analysts can understand why it was produced. A phishing detector that returns only a label may be accurate yet still difficult to trust, audit or learn from. The benchmark study [41] addressed this concern through LIME explanations for individual predictions. Subsequent research has expanded the same principle using SHAP, LIME and hybrid feature-attribution techniques. For example, the boosting-based phishing-site framework in [53] combines XGBoost with SHAP and reports 97.78% across the principal classification metrics, while [54] applies several explainability techniques to large-scale web data for feature selection and interpretation. Time-aware phishing analysis has also been combined with LIME and SHAP [52], and explainable web-phishing classification has been explored in IoT and cloud-oriented contexts [44].

The broader cybersecurity taxonomy in [48] positions explainability as part of trustworthy automation rather than as a cosmetic interface feature. Transformer-oriented work further demonstrates that explanation can accompany contextual models in phishing and adjacent spam-classification tasks [46], [47]. SEDAS follows this literature but selects a lighter mechanism suited to its strongest linear classifier. Instead of invoking a separate local explanation model, it multiplies each present TF-IDF feature by the trained classifier coefficient and surfaces the strongest positive contributions. This preserves the practical objective established by [41]—making individual decisions inspectable—while reducing runtime and implementation complexity. The resulting indicators support FR-02 and also connect technical detection to the awareness objective by showing users which terms contributed to elevated risk.

2.5 Security Awareness Training and Human Factors

Technical detection alone does not address the human vulnerabilities that social engineering exploits, motivating the awareness training component of SEDAS. A review synthesising over thirty peer-reviewed studies found that simulation-based training significantly reduces user susceptibility to phishing, particularly when training is adaptive, behaviour-aware and reinforced over time — directly supporting SEDAS's persona-based, risk-adaptive design [56]. A complementary study found that exposure to training does not automatically translate into durable behavioural change, and that feedback and engagement strategy design materially affect outcomes, with training fatigue and predictability reducing effectiveness over repeated exposure [45]. This reinforces the design rationale in Section 4.3 for immediate per-question feedback and adaptive difficulty rather than static, repetitive content, consistent with the Theory of Planned Behaviour grounding established in the Part A design.

Phishing is ultimately a socio-technical problem because even strong filtering cannot remove every situation in which a user must judge whether a message, request or link is trustworthy. This makes awareness training a complementary control rather than an optional addition to detection. The review in [56], which synthesises more than thirty studies, concludes that simulation-based security awareness can reduce phishing susceptibility, particularly when learning is reinforced and adapted to observed behaviour.

This supports the SEDAS decision to record user performance and select subsequent lesson difficulty according to a continuously updated risk score rather than delivering identical material to every participant.

Training quality also depends on how feedback is delivered. Reference [45] shows that exposure to training does not automatically produce durable behavioural change; feedback design, engagement and repeated experience influence whether users retain and apply the intended lessons. Predictable or repetitive exercises can also create fatigue, reducing attention over time. SEDAS therefore provides immediate per-question feedback and explanations so that an incorrect answer becomes a learning event at the moment the misconception occurs. Its persona-based content also allows examples to reflect the risks faced by students, staff and administrators rather than relying on one generic scenario set. The adaptive mechanism is consistent with the Theory of Planned Behaviour grounding established in Part A: the objective is not merely to increase knowledge, but to reinforce the judgement and behavioural control required when users encounter real social-engineering prompts.

2.6 Trust and Integrity Verification Mechanisms

Blockchain and hash-chaining techniques have been applied to phishing defence specifically to provide tamper-evident records of URL classifications. MLPhishChain combines machine learning classification with an Ethereum-based blockchain, using smart contracts to automate URL re-evaluation and recording classifications immutably [55]. This directly parallels SEDAS's URL verification service, and the comparison is instructive: where MLPhishChain relies on Ethereum smart contracts and wallet-based interaction, SEDAS's refinement register (R-01) adopts a SHA-256 hash-chained registry that preserves tamper-evidence — each record's hash depends on its predecessor, so any modification is cryptographically detectable — without the gas fees, wallet infrastructure or transaction latency a public blockchain requires. Cryptographic chaining alone, independent of a distributed ledger, is sufficient to make tampering mathematically detectable, at the cost of decentralised consensus, which SEDAS explicitly and transparently forgoes at MVP scope.

Phishing defence also requires confidence in the integrity of shared threat information. A URL blocklist or trust registry loses value if records can be modified without detection, because later users may rely on a falsified status. Blockchain-based approaches address this by making classification records difficult to alter retrospectively. MLPhishChain [55], for example, combines machine-learning classification with Ethereum smart contracts so that URL decisions can be recorded and re-evaluated within a distributed ledger.

SEDAS preserves the core integrity objective but implements it differently at MVP scope. Under refinement R-01, the Part A Ethereum concept was replaced by an append-only SHA-256 hash chain. Each registry entry stores the previous entry's hash and computes its own digest from the URL, verdict, reporter, timestamp and predecessor hash. Changing one historical record therefore invalidates that entry and every later link when the chain is revalidated. The result is tamper-evidence: modification is mathematically detectable through the validation endpoint. It is not equivalent to a blockchain, because there is no decentralised consensus and a sufficiently privileged database administrator could theoretically rewrite the data and recompute the entire chain. This limitation is explicit rather than hidden. The refinement was chosen because wallet management, gas costs, transaction latency and public-chain infrastructure would add substantial engineering overhead without improving the assessable MVP behaviour. R-01 therefore retains the security property required for demonstration while deferring distributed consensus to future production work.

2.7 Research Gap

Synthesising the reviewed literature alongside the base paper [41], three gaps motivate this study. First, reproducibility: consistent with [41]'s own stated motivation, much of the surveyed literature does not disclose full preprocessing and configuration detail, limiting independent verification — a gap this study

addresses through a fully disclosed, re-runnable replication. Second, integration: the reviewed studies each address detection, explainability, training or blockchain verification in isolation; none combine all four within a single evaluated system, which is the specific contribution of SEDAS. Third, the classical-versus-transformer question remains unsettled at the margins — while [49], [50] and [51] report transformer advantages, the reported margins vary considerably, and none benchmark against the specific dataset and configuration used in [41], which Section 5.5 addresses directly.

The reviewed literature reveals three connected gaps that define the contribution of SEDAS. The first is reproducibility. As recognised by the motivation of the base study [41], phishing research often depends on datasets or preprocessing decisions that are not sufficiently disclosed for independent verification. High reported scores are difficult to interpret when another team cannot recreate the same input conditions. SEDAS addresses this gap by using the identical public corpus associated with [41] and disclosing its split, seed, feature settings and evaluation protocol.

The second gap is integration. Existing studies commonly isolate one capability: email classification, explainability, awareness training, URL reputation or blockchain-based integrity. The reviewed work demonstrates the value of each component, but does not evaluate them as one connected defensive workflow. SEDAS therefore combines explainable email detection with URL verification, a tamper-evident registry, adaptive training and governance-oriented records.

The third gap concerns the remaining margin between strong classical baselines and transformer methods. Studies [49]-[51] generally report transformer advantages, but the size of those gains varies materially by corpus and experimental design. None provides an equivalent full comparison against the exact SEDAS replication configuration of [41]. Section 5.5 therefore treats DistilBERT as an exploratory contextual alternative while preserving a clear comparability caveat. Together, these gaps shift the research question from simply seeking a slightly higher accuracy number toward establishing what is reproducible, what additional optimisation still has value, and how detection can support broader social-engineering defence.

3. Methodology

This section follows the base paper's methodology sequence: research design, architecture, dataset, preprocessing, algorithms, extension experiments, transformer approach, evaluation protocol, and the refinements applied to the original design.

3.1 Research Design

The study used a two-stage experimental design: independent replication followed by systematic extension. Replication was deliberately performed first because an improvement claim is meaningful only after the underlying pipeline has been shown to reproduce the published benchmark under comparable conditions. SEDAS therefore loaded the same public corpus used by [41], recreated the TF-IDF-based classical pipeline and evaluated the paper's three methods before adding alternative models or optimisation strategies. This sequence establishes a validated baseline against which later changes can be interpreted.

The development lifecycle followed the Unified Process, with Inception and Elaboration used to review the Part A specification and formally re-baseline scope, Construction used to implement and integrate the services, and Transition used for testing, refinement and documentation. Agile practices operated within those phases: work was divided into short sprints, functionality was developed on feature branches, automated tests ran continuously, and supervisor feedback was incorporated into subsequent experiments. This combination retained the traceability of UP while allowing iterative technical decisions.

For the classical comparison, all models used the same cleaned corpus, identical 80/20 stratified split and common evaluation metrics. The four extension experiments reused the same baseline conditions so that any change in performance could be attributed to the method rather than to a different test set. DistilBERT was the exception in data volume because CPU constraints required a stratified subsample; it is therefore reported separately and not treated as an equivalent full-data comparison.

3.2 System Architecture

SEDAS — System Architecture

Microservice decomposition with independent data stores

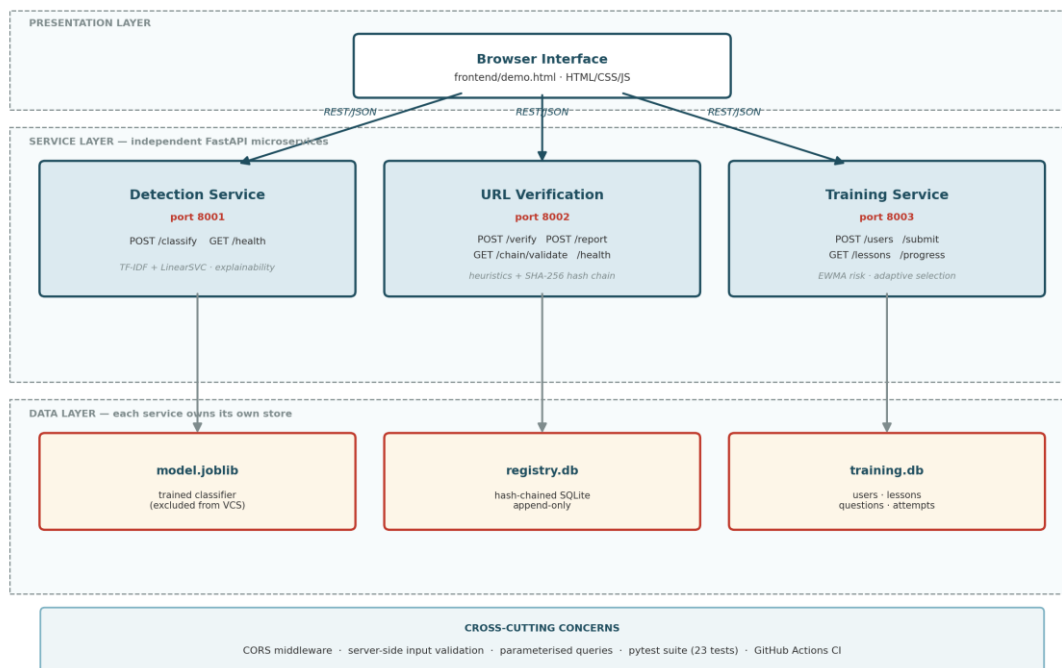


Figure 1: SEDAS system architecture — three-layer decomposition with independent per-service datastores.

SEDAS uses a three-layer architecture separating presentation, application services and data. The presentation layer is a browser interface that collects email and URL input and renders verdicts, risk scores, explanations, registry evidence and training progress. It contains no core decision logic. The service layer contains three independent FastAPI applications. The detection service performs stateless email classification using the persisted machine-learning pipeline. The URL verification service combines heuristic inspection with a write-oriented tamper-evident registry. The training service maintains user, lesson, question and attempt state and applies adaptive scoring logic. The data layer is separated by service: the detector uses a serialised model artefact, while the URL and training services maintain independent SQLite stores.

Microservice decomposition was retained because the three capabilities have different runtime characteristics and security concerns. Detection is compute-bound and stateless at request time; URL verification is comparatively write-heavy and must preserve chain integrity; training is read-heavy but maintains per-user state. Separating these concerns allows each service to be scaled, replaced, tested or secured independently, and a failure in one capability does not automatically disable the others.

The accepted MVP trade-off is operational overhead. Three processes must be started rather than one, and there is no API gateway, service discovery or orchestration layer. At this scale, however, those components would add infrastructure complexity without changing the functional evidence being assessed. Communication uses HTTP with JSON payloads throughout. FastAPI generates OpenAPI specifications automatically from the endpoint and validation models, making each service contract machine-readable and consistent with the implemented code.

3.3 Dataset

Attribute	Value	Note
Total emails	82,486	after removal of null entries
Phishing	42,891	matches base paper exactly
Legitimate	39,595	matches base paper exactly
Source corpora	Six merged	Enron, Ling-Spam, CEAS 2008, Nazario, Nigerian Fraud, SpamAssassin
Fields used	text_combined, label	subject and body merged; binary label
Availability	Public	freely available, supporting reproducibility

Table 5: Dataset composition.

The experimental dataset contains 82,486 usable emails after null records were removed. Of these, 42,891 are labelled phishing and 39,595 legitimate. The combined corpus draws from six established public sources: Enron, Ling-Spam, CEAS 2008, the Nazario phishing corpus, Nigerian Fraud and SpamAssassin. The modelling fields are `text_combined`, which merges the available subject and message body information, and a binary label distinguishing phishing from legitimate email.

The decisive reason for selecting this corpus was not simply its size. It is the dataset published alongside the benchmark study [41], which makes a direct replication legitimate. The loaded class counts were checked against the figures reported in the benchmark and match exactly: 42,891 phishing and 39,595 legitimate records. This correspondence provides stronger comparability than evaluating a similar algorithm on a different Kaggle collection and then comparing headline scores as if the tasks were equivalent. The same underlying examples allow observed differences to be discussed in relation to pipeline configuration and learning method rather than corpus composition.

Public availability also directly supports the reproducibility objective. A third party can acquire the same dataset, apply the disclosed preprocessing and fixed split, and verify the results without access to proprietary institutional mail. This addresses the transparency limitation that [41] identifies in earlier phishing research. The trade-off is external validity: the six component corpora are research collections assembled for experimentation rather than live traffic from the target institution. They may not reproduce current organisational language, user behaviour or contemporary attack distribution. That domain-shift limitation is acknowledged here and examined more fully in Section 5.8.

3.4 Data Model

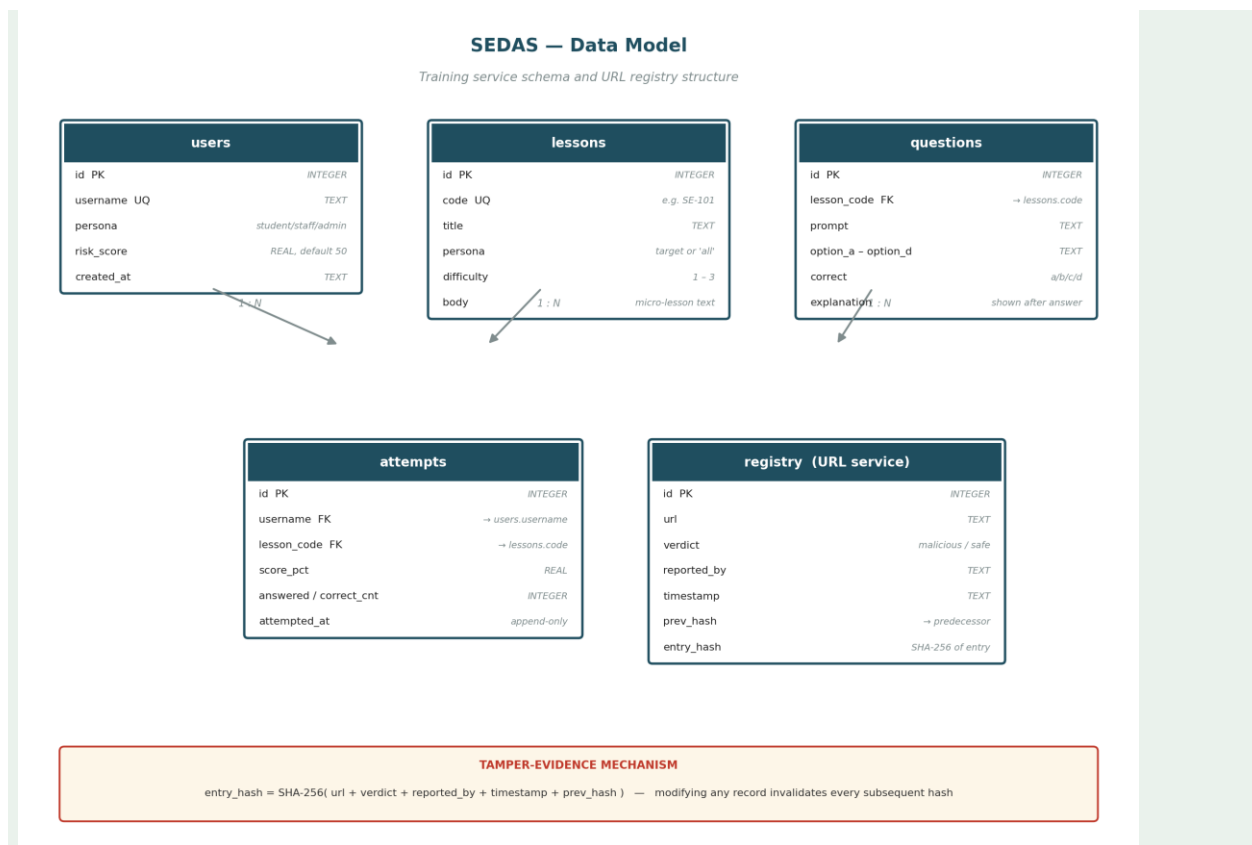


Figure 2: Entity structure for the training service and the URL verification registry.

```
entry_hash = SHA-256( url + verdict + reported_by + timestamp + prev_hash )
```

```
# The first record uses the literal string 'GENESIS' as prev_hash.
# Because each hash depends on its predecessor, altering any stored
# record invalidates that record's hash and every subsequent hash.
```

The SEDAS data model separates training state from URL-integrity records. The training service uses four relational tables. `users` stores a unique username, persona and current risk score. `lessons` stores the lesson code, intended persona, difficulty and instructional content. `questions` links quiz items to a lesson and stores the prompt, options, correct answer and explanation. The quiz endpoint deliberately projects only the prompt and answer options; the correct value and explanation remain server-side until submission so that a client cannot retrieve the answer key in advance. `attempts` records each completed quiz with the user, lesson, score and timestamp. Attempts are append-only rather than overwritten, preserving an audit trail and allowing progress or risk trends to be reconstructed.

The URL verification service uses a separate registry table containing the URL, verdict, reporter, timestamp, previous hash and entry hash. The first record uses GENESIS as its predecessor. Every subsequent entry computes its SHA-256 digest from its own contents plus the previous entry's hash. Changing one historical row therefore breaks that row's digest and the linkage of later records. The validation endpoint detects modification by walking the chain and recomputing each hash.

This mechanism provides tamper-evidence, not decentralised consensus. A sufficiently privileged actor controlling the database could theoretically rewrite records and recompute the chain. A distributed ledger or external anchoring service would be required to remove that trust point. For the MVP, the chosen model preserves the integrity property being demonstrated without the operational cost of blockchain infrastructure.

3.5 Preprocessing and Feature Extraction

The classical detection pipeline represents email text using Term Frequency-Inverse Document Frequency (TF-IDF). TF-IDF assigns greater weight to terms that are important within a particular message but less common across the corpus, reducing the influence of ubiquitous words while retaining discriminative vocabulary. The vectoriser was configured with a maximum of 50,000 features, an n-gram range of 1-2 and English stop-word removal. Including bigrams is important because phishing intent is often expressed through short constructions such as “verify account”, “click here” or “urgent action”, whose combined meaning can be more informative than either word in isolation.

The resulting matrix is high-dimensional and sparse. Every email is mapped into a 50,000-feature space, but only a small fraction of those positions are non-zero for any individual message. This representation is particularly suitable for linear text classifiers such as SVC and Logistic Regression, which can exploit many weak but consistent lexical signals efficiently. The same sparsity is relevant to interpreting the weaker XGBoost result in Section 5.4, because tree-based boosting must repeatedly partition a feature space in which most values are zero.

A separate extension experiment tested whether additional preprocessing and hand-crafted signals could add information not already captured by TF-IDF. URLs in the text were normalised to a common token, HTML tags were stripped, and six numeric features were calculated: message length, URL count, exclamation-mark count, digit count, capital-letter ratio and the number of phishing-related keywords. These features were concatenated with the TF-IDF representation and evaluated under the same held-out protocol, enabling a controlled test of whether explicit security heuristics improved the strong baseline.

3.6 Algorithms Investigated

Five classical algorithms were evaluated under the common TF-IDF pipeline. LinearSVC, Random Forest and Multinomial Naive Bayes reproduce the model families examined in the benchmark study [41]. Logistic Regression and XGBoost were added as extensions to test whether another linear probabilistic method or a sequentially boosted tree ensemble could improve on the published result. Each algorithm therefore serves a distinct comparison purpose: maximum-margin linear separation, bagged nonlinear trees, probabilistic word evidence, linear probability estimation and gradient boosting. DistilBERT, described separately in Section 3.8, provides the sixth method by replacing sparse TF-IDF features with contextual token representations.

3.6.1 Support Vector Classifier (LinearSVC)

LinearSVC implements a support vector classifier using a linear decision boundary. Training seeks a hyperplane that separates phishing and legitimate examples while maximising the margin to the nearest training points, known as support vectors. A wider margin improves robustness around the boundary. The method suits TF-IDF because text classification produces high-dimensional sparse vectors and the linear form can learn weights for many informative terms without an expensive nonlinear kernel. LinearSVC was included because SVM was the strongest classifier reported in [41], making it the primary benchmark-replication method and the reference point for later optimisation experiments.

3.6.2 Random Forest

Random Forest is an ensemble of decision trees trained on bootstrap samples of the data. Each tree learns a sequence of feature-based splits and produces a class prediction; the forest combines those predictions by majority vote. Bootstrap sampling and random feature selection reduce the tendency of one tree to overfit while allowing nonlinear interactions. SEDAS used 100 estimators with `random_state=42`, matching the benchmark's stated setup. Random Forest was included as a direct replication method from [41] and as a contrast to LinearSVC, testing whether nonlinear ensemble partitioning can exploit the same sparse TF-IDF representation effectively.

3.6.3 Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classifier commonly used for word-frequency and TF-IDF features. It estimates how likely each feature is under the phishing and legitimate classes and combines those likelihoods with class priors to choose a label. Its efficiency relies on a conditional-independence assumption: once the class is known, each word is treated as independent of the others. This simplification makes the model fast, but it cannot directly represent relationships between terms or word order. MNB was included because it is evaluated in [41]. The consequences of this assumption are examined in the precision-recall results in Section 5.2.

3.6.4 Logistic Regression (extension)

Logistic Regression is a linear probabilistic classifier. It calculates a weighted sum of TF-IDF features and transforms that score through the logistic function to estimate phishing probability. Positive weights push a message toward phishing, while negative weights support the legitimate class. The method is efficient on sparse data and its coefficients are interpretable, making it a useful contrast with LinearSVC despite different optimisation objectives. Logistic Regression was not evaluated in [41] and was added as a classical extension to test whether a probability-oriented linear model could match or exceed the maximum-margin SVC on the identical corpus and split.

3.6.5 XGBoost (extension)

XGBoost is a gradient-boosted decision-tree method in which trees are built sequentially. Each new tree focuses on correcting errors left by the current ensemble, and the final prediction combines the contributions of all trees. This can model nonlinear interactions that a linear classifier cannot represent. The SEDAS configuration used 200 estimators, maximum depth 6 and learning rate 0.1. XGBoost was not evaluated in [41]; it was added specifically to test a method beyond the benchmark. Its result also indicates whether boosted tree partitioning is well matched to the extremely sparse 50,000-dimensional TF-IDF representation used in this study.

3.7 Extension Experiments

Direct replication established that the SEDAS pipeline was correct, but replication alone could not demonstrate an original empirical contribution. Four controlled extension experiments were therefore designed to test plausible ways of exceeding the SVC baseline while holding the dataset, split and evaluation procedure constant.

Experiment 1 combined TF-IDF with additional preprocessing and six engineered security signals. URLs were normalised, HTML tags removed, and features such as URL count, capital-letter ratio and phishing-keyword count were appended. The hypothesis was that explicit structural and stylistic signals might capture information that lexical TF-IDF weights missed.

Experiment 2 created an equal-weight voting ensemble of SVC, Random Forest and Logistic Regression. Each model received one vote, with the expectation that different decision mechanisms would make different errors and that majority agreement could compensate for individual weaknesses.

Experiment 3 retained the same ensemble but weighted SVC by three votes against one each for Random Forest and Logistic Regression. Because SVC was already the strongest individual model, the hypothesis was that it should dominate ambiguous cases rather than being outvoted whenever the two weaker models agreed.

Experiment 4 tuned the SVC regularisation parameter C over seven values from 0.1 to 10.0. This tested whether the default margin-penalty trade-off was suboptimal for the corpus. Together, the experiments examine feature representation, model combination, weighting and hyperparameter sensitivity rather than relying on one arbitrary improvement attempt. Their outcomes are analysed in Section 5.4 as evidence about the practical performance ceiling of the classical pipeline. under controlled conditions.

3.8 Transformer-Based Approach

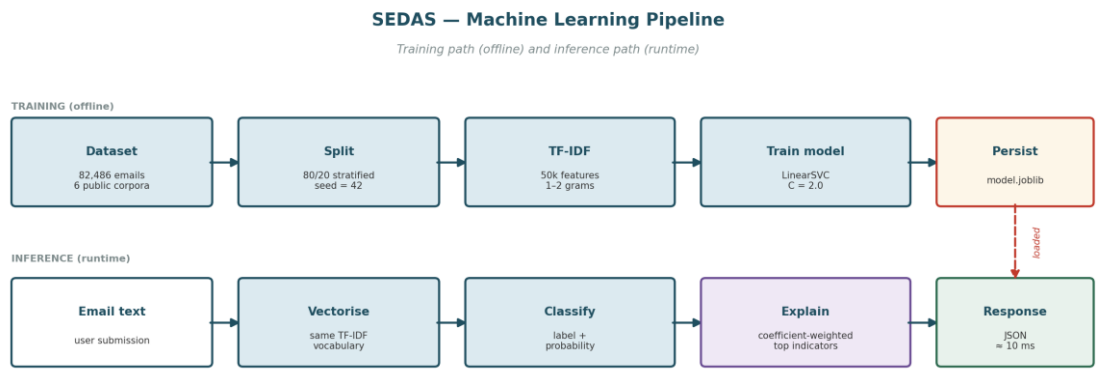


Figure 3: Machine learning pipeline — offline training path and runtime inference path.

The transformer experiment tests a fundamentally different representation of language. TF-IDF treats a message as a weighted collection of terms and short n-grams; most long-range word order and contextual relationships are discarded. Transformers instead encode each token in relation to the surrounding sequence, allowing the model to distinguish how identical words function in different contexts. For example, “your account was verified successfully” and “verify your account immediately at this link” share several terms, but the second sequence expresses an urgent credential-lure pattern whose meaning depends on ordering and surrounding language.

DistilBERT was selected because it is a distilled, computationally lighter version of BERT that retains contextual modelling while reducing memory and training cost. The available environment was CPU-only, so the experiment was intentionally preliminary rather than a full-corpus benchmark. A stratified sample of 6,000 emails was drawn, containing 3,120 phishing and 2,880 legitimate messages. This produced 4,800 training examples and 1,200 held-out test examples using an 80/20 split with random_state 42. Inputs were truncated or padded to 128 tokens, batch size was 8, training ran for two epochs, and the learning rate was 2e-5.

These settings balance context length against feasible CPU execution. They also impose an important limitation: the transformer received only about 7.3% of the full corpus and cannot be directly compared with classical models trained on 65,988 emails. Full-data fine-tuning would require GPU resources and is therefore deferred to future work.

3.9 Refinements from the Part A Design

Ref	Refinement	Justification
R-01	Ethereum smart contracts → SHA-256 hash-chained registry	Preserves tamper-evidence without gas fees, wallet infrastructure or block latency; Ethereum retained as future work
R-02	Browser extension → web form and API	Extension store publishing and cross-browser testing exceed the timeline; verification capability fully demonstrated
R-03	Live LMS/SIEM/SSO integration → documented API contracts	No institutional access available to a student team

Ref	Refinement	Justification
R-04	$F1 \geq 0.95$ target → honest gap analysis approach	Model quality depends on data volume and compute unavailable in the unit
R-05	Latency < 200 ms (GPU) → < 2,000 ms (CPU)	GPU compute out of budget; latency measured and reported so the production path is evidenced
R-06	Kubernetes and Kafka → direct REST between services	Scale-out infrastructure does not change functional behaviour at MVP scale; microservice decomposition retained
R-07	Two institutional case studies → single simulated scenario	Real pilots are outside unit scope and would require ethics approval
R-08	SMS/IM interception → documented future work	Separate delivery channel with material engineering cost

Table 6: Refinement register — changes from the Part A specification with justification.

The Part A design was treated as a specification to be critically reviewed rather than as an immutable implementation contract. Before Construction began, the team consolidated the requirements, evaluated feasibility and formally recorded eight refinements in the specification review memorandum. These changes were discussed as part of the project governance process and became the Part B baseline. Refinement therefore represents an expected design outcome: the purpose of implementation is partly to test whether an earlier architecture remains appropriate when real constraints, resources and assessment objectives are applied.

R-01 was one of the most significant changes. Part A proposed Ethereum-compatible smart contracts for the URL trust layer. Part B replaced this with a SHA-256 hash-chained append-only registry. The preserved design intent is tamper-evidence: modifying a historical record breaks the cryptographic chain and is detectable during validation. What was given up is decentralised consensus and the resistance to a fully privileged administrator rewriting and rehashing the database. The trade-off avoided gas fees, wallet management and blockchain transaction latency while retaining the integrity property needed for the MVP.

R-05 similarly re-baselined latency from an aggressive GPU-oriented target to less than 2,000 ms on commodity CPU hardware. The design intent—interactive, operationally useful response—was preserved, while dedicated GPU infrastructure was given up. Measured classification and verification times remained in the single- to low-double-digit millisecond range, comfortably exceeding the revised target. These refinements therefore narrowed infrastructure ambition without weakening the core behaviours being evaluated.

3.10 Evaluation Protocol

Parameter	Value
Split	80% training / 20% held-out test, stratified by class
Training set	65,988 emails
Test set	16,498 emails (never seen during training)
Random seed	42 (fixed, for full reproducibility)
Metrics	Accuracy, precision, recall, F1, false-positive rate, confusion matrix, latency

Table 7: Evaluation protocol.

All classical experiments used an 80/20 stratified hold-out design. From the 82,486-email corpus, 65,988 records were assigned to training and 16,498 to the test set. The test examples were not used to fit the models. This separation is essential because evaluating on training data would primarily measure memorisation of examples already seen rather than the ability to generalise to unseen email. Stratification preserved the phishing-to-legitimate class ratio in both partitions, preventing an accidental class-balance change from influencing one model more than another. A fixed random seed of 42 was used so that repeated executions produce the same allocation, making model differences and third-party replication verifiable.

Evaluation used accuracy, precision, recall, F1 score, false-positive rate, confusion matrices and measured latency. Accuracy asks what proportion of all emails were classified correctly. Precision asks, among emails labelled phishing, how many were actually phishing. Recall asks how many of the genuine phishing emails were detected. F1 is the harmonic mean of precision and recall and is used as the headline comparison because it penalises a model that appears strong by maximising one at the expense of the other. The confusion matrix exposes the absolute counts of true negatives, false positives, false negatives and true positives, making the direction of errors visible. False-positive rate is reported separately because unnecessary blocking of legitimate communication has a direct deployment cost and can erode trust in warnings. Latency measures whether the model can return its result fast enough for interactive use. Together these metrics evaluate both statistical quality and operational suitability rather than reducing performance to one aggregate number.

4. System Implementation and Deployment

The base paper's Section 4 covers web deployment of its model. This section covers SEDAS's implementation, which extends beyond a single deployed classifier to four integrated components.

4.1 Detection Service

Endpoint	Method	Behaviour
/classify	POST	Accepts email text; returns label, risk score 0-100, top contributing indicators, measured latency
/health	GET	Returns service status for monitoring

Table 8: Detection service endpoints (port 8001). Measured latency $\approx$ 7-11 ms.

The detection component is exposed as a FastAPI service on port 8001. Its primary contract is POST /classify, which accepts email text and returns the predicted label, a 0-100 phishing risk score, the most influential indicators and measured request latency. GET /health provides a lightweight status endpoint for monitoring and automated testing. Validation uses typed request models so malformed input is rejected before classification executes.

Explainability is derived directly from the trained linear model. The submitted message is transformed by the same TF-IDF vectoriser used during training. For every feature present in the sparse vector, its TF-IDF value is multiplied element-wise by the corresponding classifier coefficient. The sign indicates whether that feature pushes the decision toward phishing or legitimate, while the magnitude reflects the strength of its contribution for the specific message. SEDAS sorts the positive contributions and returns the five largest as visible indicators.

This matters because a security decision should not require blind trust in a score. A user seeing terms such as “verify”, “account”, “click” or “immediately” can relate the warning to message content and learn a reusable pattern. The same output gives an analyst a rapid sanity check on whether the model relies on plausible evidence. It therefore satisfies FR-02 without the overhead of a separate explanation model. Measured classification latency was approximately 7-11 ms on CPU hardware, far below the NFR-03 target of 2,000 ms and suitable for interactive use.

4.2 URL Verification Service

Heuristic indicator	Rationale
No HTTPS	Unencrypted transport for a credential-collecting page
Raw IP address as host	Legitimate services almost always use domain names
'@' in URL	Everything before '@' is discarded by browsers, hiding the true destination
Multiple hyphens in domain	Common in lookalike domains imitating a legitimate brand
Excessive subdomain depth	Places a trusted-looking name in a subdomain of an attacker domain
High-risk TLD	Certain TLDs are disproportionately represented in abuse data
Known URL shortener	Destination cannot be inspected before clicking
Abnormal length	Long URLs obscure the destination in address bars
Credential-lure keywords	Terms such as login, verify, secure, account, password

Table 9: URL heuristic indicators (port 8002). Measured latency $\approx$ 1.4-9.3 ms.

The URL verification service uses two complementary layers because neither heuristics nor prior reports are sufficient alone. The first layer performs immediate structural analysis of an untrusted URL. It checks indicators including missing HTTPS, a raw IP host, use of “@” to obscure the destination, multiple hyphens, excessive subdomain depth, high-risk top-level domains, known shorteners, abnormal length and credential-lure terms such as login, verify or password. Each triggered rule contributes to a risk score and a plain-language finding. This provides a verdict even when the URL has never been seen before.

The second layer is a local trust registry. Reports are append-only and stored with URL, verdict, reporter, timestamp, previous hash and entry hash. For each new row, SHA-256 is calculated over the record contents plus the preceding record's hash; the first record uses GENESIS. Because every digest depends on its predecessor, changing a historical row breaks that link and all subsequent validation. GET /chain/validate walks the complete sequence and recomputes the hashes, providing evidence that the stored history has not been altered without detection.

Verdict resolution combines the two layers. A URL previously recorded as malicious returns blocked regardless of its current heuristic score. A URL recorded as safe returns trusted when its heuristic score remains below the suspicious threshold. An unreported URL at or above the threshold returns suspicious, while an unreported URL below it returns low_risk. The registry therefore supplies organisational memory, while the heuristic layer addresses first-seen links and protects against blindly trusting an old safe label when the structure is concerning.

The design deliberately provides tamper-evidence rather than decentralised consensus. Unlike MLPhishChain [55], which uses Ethereum smart contracts, SEDAS has no distributed validator set or public-chain immutability. A fully privileged operator could theoretically rewrite and rehash the database. In return, the MVP avoids wallet infrastructure, gas fees and blockchain latency while demonstrating the integrity requirement through standard cryptography. Measured verification latency of approximately 1.4-9.3 ms shows that the two-layer design remains comfortably within NFR-02.

4.3 Awareness Training Service

```
# Risk update - exponentially weighted moving average
new_risk = (1 - ALPHA) * current_risk + ALPHA * (100 - score_pct)
# ALPHA = 0.4

# Risk bands: low < 33 | medium < 66 | high >= 66
# Target difficulty: high risk -> 1, medium -> 2, low -> 3
```

The awareness service converts detection-oriented security into an adaptive learning process. Users are registered with a persona—student, staff or administrator—and lessons can be targeted to a specific persona or made available to all. When a lesson quiz is requested, the API returns only the question prompts and answer options; the stored correct answer and explanation remain server-side until submission. After a response is submitted, the service scores the attempt and returns immediate per-question feedback. This design follows the human-factor evidence in [45], which shows that the structure and timing of feedback influence whether phishing training produces meaningful behavioural learning rather than short-lived exposure.

Each completed attempt is retained and contributes to a continuously updated risk score. SEDAS uses an exponentially weighted moving average: $\text{new_risk} = (1 - \text{ALPHA}) \times \text{current_risk} + \text{ALPHA} \times (100 - \text{score_pct})$, with ALPHA set to 0.4. The current quiz therefore influences 40% of the update while 60% of the previous risk estimate is retained. This weighting makes the score responsive to recent performance without allowing one unusually good or bad attempt to completely redefine a user's risk profile. Risk is divided into three bands: low below 33, medium below 66, and high at 66 or above.

Lesson selection intentionally inverts difficulty relative to risk. High-risk users receive difficulty 1 content, medium-risk users receive difficulty 2, and low-risk users receive difficulty 3. This avoids treating poor performance as a reason to make learning harder. A struggling learner is first reinforced on foundational warning signs, while users

demonstrating stronger judgement progress to subtler scenarios. The approach is therefore remedial rather than punitive.

Adaptive selection also includes fallback behaviour. If no unfinished lesson exists at the exact target difficulty, the service searches other eligible lessons instead of returning no content. This ensures that a user continues learning while suitable material remains available. Attempts are append-only, so the service preserves a history for progress tracking and later governance analysis as required by FR-05 and FR-06.

4.4 Governance and Interface Layer

The browser interface provides a single demonstration surface for the three SEDAS capabilities. Email text can be submitted to the detection service, with the resulting label, risk score, explanatory indicators and latency rendered to the user. URLs can be submitted to the verification service, which displays the resolved verdict, heuristic findings, risk score and registry proof where a prior report exists. Training progress is exposed through the training APIs, including attempts, current risk score, risk band and the next adaptive lesson.

The interface is intentionally treated as supporting implementation rather than as the research contribution. This follows supervisor guidance that web-development effort should not displace machine-learning execution, evaluation and technical reasoning. The contribution therefore lies in the validated services and evidence they produce, not in visual styling.

The governance layer is partly implemented at the data level even though a full role-based analytics dashboard was deferred. Append-only training attempts, user risk scores, lesson completion data, registry reports and validation records are captured in structured form for later querying or visualisation. FR-07 is therefore partial rather than absent: the underlying governance evidence exists, while the production dashboard and role-based access experience remain future work.

4.5 Technology Stack

Layer	Technology	Purpose
Runtime	Python 3.14	Application language across all services
Web framework	FastAPI + uvicorn	REST endpoints, validation, automatic OpenAPI generation
Validation	Pydantic	Typed models; rejects malformed input before handler execution
Machine learning	scikit-learn, xgboost	TF-IDF, LinearSVC, RandomForest, MultinomialNB, LogisticRegression, XGBoost
Transformer	PyTorch, Hugging Face Transformers	DistilBERT fine-tuning
Persistence	SQLite 3, joblib	Service datastores and model serialisation
Integrity	hashlib (SHA-256)	Registry chain hashing
Frontend	HTML, CSS, JavaScript	Demonstration interface; no build step
Testing and CI	pytest, GitHub Actions	23 automated tests, executed on every push

Table 10: Technology stack by layer. No component requires a GPU.

4.6 Security Controls

4.6.1. Methodology and Scope

This assessment reviews SEDAS against each of the ten OWASP categories through manual source code review of all three services, informed by the existing automated test suite (23 tests, Section 4.7 of the Technical Manual) where a test provides direct evidence for a control. It does not include automated dependency scanning, dynamic penetration testing, or a formal threat model, which are noted as limitations of the assessment itself in Section 4.

Each finding is rated using four severity levels, defined for this MVP-scope system as follows.

Rating	Meaning at this project's scope
High	Exploitable with no special access, with a realistic and material impact (data exposure, integrity loss, denial of service). Should block a production deployment decision.
Medium	Exploitable but requires a specific condition, or has a bounded impact. Acceptable at MVP/demonstration scope with the gap documented, but should be resolved before any production use.
Resolved	The category was assessed and a control is implemented and verified by an automated test.
N/A	The category does not apply to this system's architecture, with justification given.

Table 1: Severity rating definitions used in this assessment.

4.6.2. Findings by Category

A01:2021 — Broken Access Control

Applicability: Applicable — all three services expose endpoints without any authentication layer.

No authentication or role-based access control is implemented at MVP scope (FR-08 is explicitly deferred in the Part A refinement register). Any party able to reach a service endpoint can invoke it — including submitting URL reports, registering training users, or querying another user's progress by username. There is no concept of a session, a role, or an authorisation check anywhere in the codebase.

Severity: **High**

Evidence: Manual code review of all three `serve.py` files; no auth-related middleware, dependency injection, or decorator present on any route.

Remediation: Implement an authentication layer (institutional SSO is the intended production path per the Part A design) with role-based access control distinguishing student, staff and administrator personas, particularly restricting POST `/report` and GET `/progress/{username}` to the authenticated user or an administrator role.

A02:2021 — Cryptographic Failures

Applicability: Applicable — data in transit and at rest.

All three services run over plain HTTP on localhost, appropriate for local development but not for any network-exposed deployment. SQLite datastores are not encrypted at rest; a party with filesystem access to `registry.db` or `training.db` can read all records directly. The SHA-256 hash chain in the URL registry provides integrity (tamper-evidence), which is a distinct property from confidentiality, and should not be mistaken for encryption — this distinction is stated explicitly in the Technical Manual, Section 4.2.

Severity: **Medium**

Evidence: Service startup configuration reviewed; no TLS termination configured; SQLite files reviewed and confirmed unencrypted.

Remediation: Terminate TLS at a reverse proxy (or via `uvicorn`'s own TLS support) before any deployment beyond localhost. Consider SQLCipher or filesystem-level encryption for the datastores if they will hold data beyond synthetic test records.

A03:2021 — Injection

Applicability: Applicable — all three services accept user-supplied text that reaches a database query.

All database access uses parameterised queries via the `sqlite3` module's placeholder syntax; no request data is concatenated into a SQL string anywhere in the codebase. This was directly verified by an adversarial test rather than by code review alone.

Severity: **Resolved**

Evidence: Test TS-01 (`test_security.py`) submits a URL payload containing SQL metacharacters, including a DROP TABLE statement, to POST `/report`, then confirms the registry table still exists, the hash chain still validates, and the payload is retrievable as literal text — proving the input was never executed as SQL. Test passing as of the current suite run (23/23 passed).

Remediation: No action required for the current attack surface. If future development introduces raw query construction (for example, a dynamic search or filter feature), the same parameterisation discipline must be maintained and covered by an equivalent adversarial test.

A04:2021 — Insecure Design

Applicability: Applicable — assessed at the architecture level.

Positively, the URL verification design applies defence in depth — heuristic analysis and a tamper-evident registry as two independent layers — and the training module's EWMA risk model is deliberately resistant to single-event manipulation. Negatively, no formal threat model was produced during design (the Part A specification did not

include one), and no rate limiting exists on any endpoint, meaning the design does not itself resist high-volume abuse such as mass false URL reports intended to poison the registry.

Severity: **Medium**

Evidence: Architecture and refinement register reviewed (Technical Manual, Sections 2 and 3.9); no rate-limiting middleware present in any service.

Remediation: Produce a lightweight threat model (STRIDE is sufficient at this scale) as part of future work, and add request rate limiting, particularly on POST /report, to prevent registry poisoning through volume.

A05:2021 — Security Misconfiguration

Applicability: Applicable — CORS configuration and default framework exposure.

CORS middleware is configured with allow_origins set to a wildcard across all three services, permitting requests from any origin. This was a deliberate choice to support the local demonstration interface, which is opened directly as a file rather than served from a fixed origin, and it is already documented as a known limitation in the Technical Manual. Additionally, FastAPI's automatic /docs and /openapi.json endpoints are enabled by default on all three services with no restriction, exposing the full API surface and schema to anyone who can reach the service.

Severity: **Medium**

Evidence: CORSMiddleware configuration reviewed in all three serve.py files; confirmed allow_origins=["*"]. /docs endpoints confirmed reachable without restriction during manual testing (see Technical Manual evidence, Section 5).

Remediation: Before any deployment beyond local demonstration, restrict allow_origins to the specific frontend origin, and disable or authentication-gate the /docs and /openapi.json endpoints in production configuration (FastAPI supports disabling these via docs_url=None).

A06:2021 — Vulnerable and Outdated Components

Applicability: Applicable — the project depends on a substantial third-party stack.

The project depends on FastAPI, scikit-learn, xgboost, PyTorch, Hugging Face Transformers, pandas and their transitive dependencies. No automated dependency vulnerability scanning (such as pip-audit or GitHub's Dependabot alerts) is currently configured in the CI pipeline, so a known CVE in a dependency would not be automatically flagged.

Severity: **Medium**

Evidence: GitHub Actions workflow file (.github/workflows/ci.yml) reviewed; confirmed it runs pytest only, with no dependency audit step.

Remediation: Add a pip-audit or equivalent step to the CI workflow, and enable GitHub Dependabot alerts on the repository so dependency vulnerabilities surface automatically rather than requiring manual review.

A07:2021 — Identification and Authentication Failures

Applicability: Applicable — directly related to A01, assessed separately per the OWASP category split.

Because no authentication exists (A01), there is correspondingly no session management, no credential storage, and no password policy to assess — the category's usual failure modes (weak passwords, session fixation, credential stuffing) do not yet apply because there is no identification mechanism for them to attack. This should not be read as a strength: it reflects an absent control rather than a correctly implemented one, and becomes directly applicable the moment authentication is added.

Severity: **High**

Evidence: Same review as A01; no login, session or credential-handling code exists in any service.

Remediation: When authentication is implemented (see A01 remediation), apply standard protections at the same time: hashed and salted credential storage (or, preferably, delegation to institutional SSO so SEDAS never handles credentials directly), session expiry, and protection against brute-force attempts.

A08:2021 — Software and Data Integrity Failures

Applicability: Applicable — both a strength and a residual gap.

The URL verification registry directly addresses this category: each entry's hash is computed over its own content plus its predecessor's hash, so any modification to a stored record is cryptographically detectable via GET /chain/validate. This is a genuine, tested integrity control. Separately, the trained detection model is loaded via `joblib.load()`, which uses Python's pickle format internally; loading a maliciously crafted model file can execute arbitrary code, so the model artefact must be trusted. It is currently excluded from version control and produced only by the project's own training script, but there is no cryptographic signature verifying the loaded file has not been tampered with before it is loaded at service startup.

Severity: **Medium**

Evidence: Registry chain design reviewed and confirmed via test TS-06, which writes multiple entries and revalidates the full chain. `serve.py` in the detection service reviewed; confirmed `joblib.load()` is called on `model.joblib` with no integrity check beforehand.

Remediation: For the registry: no action required, this control is implemented and tested. For the model artefact: compute and verify a checksum of `model.joblib` before loading it in `serve.py`, and document that the model file must only ever originate from the project's own `train.py` script.

A09:2021 — Security Logging and Monitoring Failures

Applicability: Applicable — assessed against what a deployed system would need.

Each service exposes a /health endpoint reporting basic operational status, which supports uptime monitoring. However, there is no structured audit logging of security-relevant events — failed validation attempts, repeated 404s against unknown users, or a burst of URL reports from the same source are not logged or surfaced anywhere. In its current state, an attack attempting any of the abuse patterns identified elsewhere in this assessment (registry poisoning, endpoint enumeration) would leave no record for review.

Severity: **High**

Evidence: All three `serve.py` files reviewed for logging calls; only `uvicorn`'s default access log exists, which records requests but not security-relevant interpretation of them.

Remediation: Add structured application-level logging for security-relevant events specifically (validation failures, unknown-resource lookups, report submissions), and route it somewhere reviewable rather than only the console — even a rotating local log file would materially improve this before any deployment.

A10:2021 — Server-Side Request Forgery (SSRF)

Applicability: Not applicable to the current design.

SSRF requires a server to make an outbound request to a URL supplied by the client. The URL verification service does not do this: `/verify` performs string-pattern heuristic analysis on the submitted URL (checking for HTTPS absence, IP-literal hosts, hyphenation, keyword presence, and so on) and a registry lookup, but at no point does the service itself issue an HTTP request to the submitted URL. There is therefore no vector by which a malicious URL could cause the server to reach an internal resource on its behalf.

Severity: **N/A**

Evidence: Confirmed by reviewing `services/url-verify/serve.py` in full; no outbound HTTP client call (`requests`, `httpx`, or similar) is present in the verification code path.

Remediation: None required under the current design. If a future feature fetches URL content directly — for example, to inspect a landing page — this category becomes applicable and must be reassessed, with outbound requests restricted to prevent access to internal network ranges.

4.6.3. Summary of Findings

Category	Severity	One-line status
A01 Broken Access Control	High	No authentication or RBAC implemented
A02 Cryptographic Failures	Medium	No TLS, no encryption at rest — acceptable at local-dev scope only
A03 Injection	Resolved	Parameterised queries; verified by adversarial test TS-01
A04 Insecure Design	Medium	No formal threat model; no rate limiting
A05 Security Misconfiguration	Medium	Permissive CORS; API docs unrestricted
A06 Vulnerable Components	Medium	No automated dependency vulnerability scanning
A07 Authentication Failures	High	No identification mechanism exists yet (linked to A01)
A08 Software/Data Integrity	Medium	Registry hash chain resolved; model file lacks load-time integrity check
A09 Logging & Monitoring	High	No security-relevant audit logging beyond basic health checks
A10 SSRF	N/A	No outbound server-side request exists in the design

Table 2: Summary across all ten OWASP categories — 3 High, 5 Medium, 1 Resolved, 1 Not Applicable.

4.6.3.1 Overall posture

SEDAS demonstrates genuine security engineering in the areas it addresses directly: injection is resolved and independently verified rather than merely claimed, and the tamper-evident registry is a real, tested integrity control that goes beyond what the base paper [41] implements. The three High-severity findings — access control, authentication, and logging — are consistent with one another: they all stem from the single, explicitly documented MVP-scope decision not to implement production authentication (refinement register, deferred FR-08), rather than three unrelated failures. This is a defensible scope decision for a capstone artefact, provided it is stated openly rather than discovered by the assessor, which this document ensures.

4.6.4. Prioritised Remediation Roadmap

#	Action	Addresses	Effort
1	Implement authentication with role-based access control (institutional SSO as the production target)	A01, A07	High — future work
2	Add structured security-event logging (validation failures, unknown-resource access, report submissions)	A09	Medium
3	Restrict CORS to known origins; disable or gate /docs and /openapi.json outside development	A05	Low
4	Add rate limiting to POST /report and other write endpoints	A04	Medium
5	Add pip-audit / Dependabot to the CI pipeline	A06	Low
6	Add a checksum verification step before loading model.joblib	A08	Low
7	Terminate TLS before any network-exposed deployment	A02	Medium — deployment-time

Table 3: Remediation actions ordered by severity addressed, not necessarily by implementation order.

Items 3, 5 and 6 are low-effort and are recommended for completion before submission where time allows, since they meaningfully strengthen the Specialisation Implementation Depth evidence without requiring architectural change. Items 1, 2, 4 and 7 are appropriately scoped as future work and are recorded as such in Section 6.2 of the Final Project Report.

4.6.5. Limitations of This Assessment

- This is a manual, code-level review, not a dynamic penetration test or an automated static analysis scan. Vulnerabilities not visible through manual inspection of the current codebase may exist.
- No dependency vulnerability scan was run as part of this assessment; the absence of known CVEs in the current dependency versions has not been verified (this is itself finding A06).
- The assessment covers the three backend services only. The frontend interface was not assessed, on the basis that per supervisor guidance it is not part of the assessed research contribution; it should be included in any pre-deployment review.
- Findings reflect the system at version 0.1.0. Any subsequent code change should be re-assessed against this document rather than assumed to remain accurate.

4.7 Testing and Quality Assurance

Suite	Test IDs	Coverage	Requirements
Detection	TD-01..TD-05	Health, phishing detection, legitimate pass-through, explainability, latency	FR-01, FR-02, FR-09, NFR-03
URL verification	TU-01..TU-03	Heuristic flagging, report-then-blocked with proof, chain integrity	FR-03, FR-04
Training	TT-01..TT-06	Scoring, EWMA risk, banding, adaptive selection, fallback	FR-05, FR-06
Security	TS-01..TS-06	Injection, malformed input, unknown resources, answer confidentiality, health, chain integrity	NFR-05

Table 12: Consolidated test matrix — 23 tests, all passing.

4.7.1 Testing strategy

Testing was organised as a layered, requirement-traced process rather than as a final manual check. Each module has API- or unit-level tests with stable identifiers: TD for detection, TU for URL verification, TT for training and TS for security. The identifiers map to functional or non-functional requirements so that a passing test is evidence for a specific behaviour. The final suite contains twenty-three tests across the four areas.

Both success and failure paths are exercised. Detection tests cover health, phishing and legitimate cases, explanation output and latency. URL tests cover suspicious heuristics, report-then-block behaviour and chain validation. Training tests cover scoring, risk updates, band selection, adaptive lesson selection and fallback behaviour. Security tests deliberately submit malformed data, unknown resources and injection-like values. This negative testing matters because a happy-path-only suite can confirm expected input while saying little about behaviour under misuse, invalid requests or boundary conditions.

GitHub Actions runs the suite on every push and pull request so regression feedback is continuous. The repository intentionally excludes the large dataset and trained model artefact. Tests that genuinely require the locally trained model therefore skip cleanly when it is absent, allowing CI to validate imports and the harness without pretending the model exists. Model-free URL, training and security suites execute fully in CI. This balances repository hygiene and reproducibility: code and evaluation procedures remain versioned, while generated artefacts are rebuilt locally when required.

4.7.2 Security testing

The dedicated security suite verifies several controls with executable evidence. TS-01 submits a URL containing SQL metacharacters rather than a normal address. After the request, the test confirms that the registry remains accessible and that the hash chain still validates. The important result is not merely that the request returned a response; it shows that parameterised queries treated the submitted characters as data rather than allowing them to alter the SQL command or destroy the underlying table.

TS-04 checks answer confidentiality in the awareness service. It requests a quiz and asserts that neither the correct answer nor the explanation field is present in the response. This prevents a client from retrieving the answer key before submitting an attempt and preserves the validity of training scores.

TS-06 tests integrity across successive writes. Multiple registry entries are created and the complete chain is then revalidated. A valid result demonstrates that the previous-hash linkage and current-entry digest remain consistent after repeated append operations, rather than working only for a single isolated record. Together, these tests provide concrete evidence for injection resistance, controlled information disclosure and tamper-evident persistence.

4.7.3 Defect identified through testing

Expanding the test suite from eight to twenty-three tests exposed a defect in the test harness itself. Both the URL verification and training services define a module named `serve`. Because Python caches imported modules by name, whichever service was imported first was returned for both, causing the URL verification suite to execute silently against the training service and return HTTP 404 responses for endpoints that do not exist there. The cause was identified by observing that the security suite exercised the same URL endpoints successfully, because it loaded modules explicitly by file path using `importlib` rather than by module name. Applying the same explicit loading approach to the URL verification suite resolved the failures, after which all twenty-three tests passed.

The defect found during suite expansion demonstrates why testing is a discovery mechanism rather than a ceremonial confirmation step. The failing URL tests initially suggested that the service endpoints were broken, but comparison with the passing security suite revealed a different cause: both the URL and training services used a module named `serve`, and Python's module cache returned whichever implementation had been imported first. The tests were therefore exercising the wrong application instance. Loading each service explicitly by file path with `importlib` removed the namespace collision and restored all twenty-three tests to passing status. The wider lesson is relevant to any multi-service Python repository: identical module names can create silent coupling inside a shared test process even when the services run correctly as separate processes. The same explicit-loading pattern was applied consistently to the affected suites so future test order cannot reintroduce the error.

5. Results and Discussion

This section presents the empirical results of the study. Six classification methods were evaluated on the dataset used by the base paper [41], comprising 82,486 labelled emails split 80/20 into 65,988 training and 16,498 held-out test instances, stratified by class with a fixed random seed of 42. Three methods replicate those evaluated in [41]; two are classical extensions; one is a transformer evaluated at preliminary scale. Four further optimisation experiments tested whether the replicated benchmark could be exceeded.

5.1 Benchmark Replication Results

Method	SEDAS F1	Paper [41] F1	Difference	Evaluated in [41]
SVC / LinearSVC	99.16%	99.0%	+0.16	Yes — best in [41]
Random Forest	98.71%	98.0%	+0.71	Yes
Logistic Regression	98.65%	—	—	No — extension
XGBoost	97.56%	—	—	No — extension
Multinomial Naive Bayes	97.47%	98.0%	−0.53	Yes

Table 13: F1 performance of all classical methods against the base paper's reported figures.

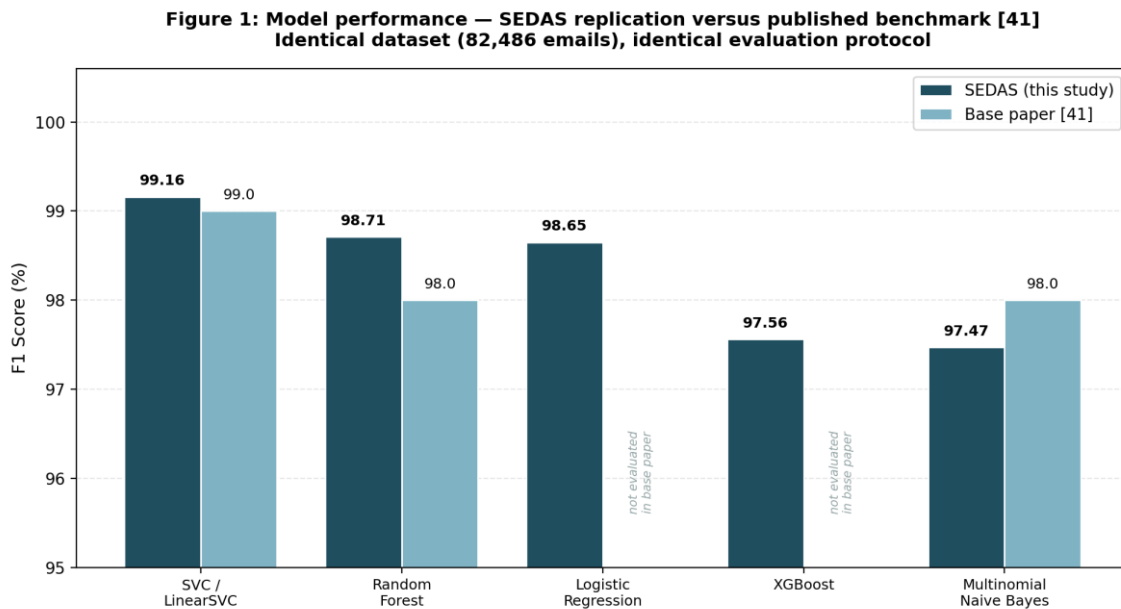


Figure 4: Model performance — SEDAS replication versus published benchmark [41].

Table 13 shows that LinearSVC is the strongest classical method in the SEDAS evaluation, matching the central conclusion of the benchmark paper rather than contradicting it. SEDAS achieved 99.16% F1 for SVC compared with the published 99.0%. Random Forest reached 98.71% against 98.0% in [41], while Multinomial Naive Bayes reached 97.47% against the published 98.0%. Thus, two of the three replicated methods are numerically above the reported benchmark figures and one is modestly below.

These differences should not be interpreted as proof that the SEDAS implementations are superior or inferior. The base paper reports rounded summary metrics but does not fully disclose every preprocessing detail needed to reconstruct an identical feature matrix. Small changes in tokenisation, stop-word handling, library defaults or implementation version can produce small score differences even when the same corpus and model family are used. In addition, [41] does not provide fold-level results, confidence intervals or repeated-run variance. Statistical significance testing between the two studies is therefore not possible from the published summaries alone. The

defensible conclusion is that the SEDAS results fall within the same high-performance range and independently confirm SVC as the strongest method among the replicated classical models.

The two classical extensions do not change that conclusion. Logistic Regression achieved 98.65% F1 and XGBoost 97.56%, both below the base paper's best SVM result. This is important because it prevents the project from treating the addition of another algorithm as a contribution by itself. Instead, the benchmark replication established a validated baseline and the weaker extension results motivated the controlled optimisation experiments in Section 5.4. The contribution therefore emerges from what those further experiments reveal about the limit of the existing representation rather than from a marginal replication delta.

5.2 Metric-Level Analysis

Metric	SVC (ours)	SVC [41]	RF (ours)	RF [41]
Accuracy	99.12%	99.1%	98.66%	98.4%
Precision	99.13%	99.0%	98.80%	98.0%
Recall	99.18%	99.0%	98.62%	99.0%
F1	99.16%	99.0%	98.71%	98.0%

Table 14a: Metric comparison for SVC and Random Forest.

Metric	MNB (ours)	MNB [41]	Difference
Accuracy	97.41%	97.8%	-0.39
Precision	99.06%	97.0%	+2.06
Recall	95.93%	99.0%	-3.07
F1	97.47%	98.0%	-0.53

Table 14b: Multinomial Naive Bayes — note the precision-recall divergence.

Figure 2: Metric-level comparison across the three replicated methods

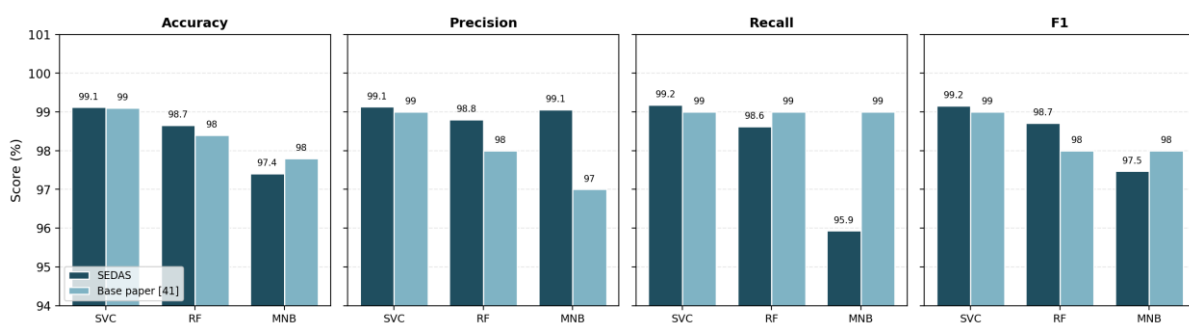


Figure 5: Metric-level comparison across the three replicated methods.

Figure 3: Confusion matrices on the 16,498-email held-out test set

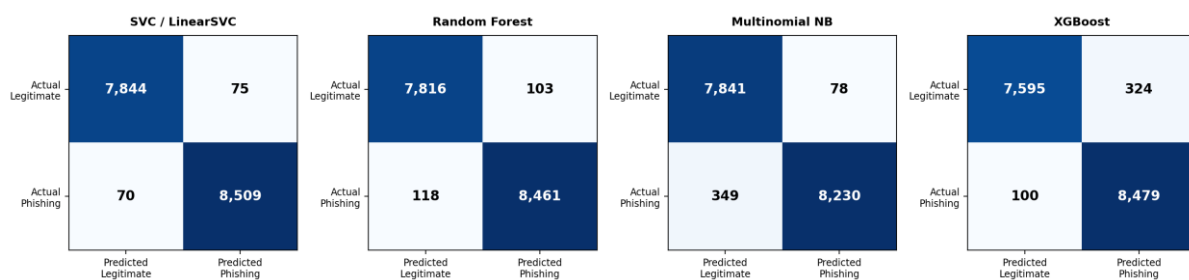


Figure 6: Confusion matrices on the 16,498-email held-out test set.

The metric-level results reveal a more informative pattern than F1 alone. For SVC, accuracy, precision, recall and F1 all remain close to 99%, producing a balanced error profile. Random Forest is also comparatively balanced, though at a slightly lower level. Multinomial Naive Bayes is different. Its precision is 99.06%, which is 2.06 points above the 97.0% reported for MNB in [41], yet its recall is only 95.93%, 3.07 points below the published 99.0%. The lower recall drives its F1 to 97.47% despite the very high precision.

Operationally, this means the SEDAS Naive Bayes model is conservative. When it labels an email as phishing, that decision is usually correct, so few legitimate messages are unnecessarily flagged. However, it fails to identify a substantially larger number of genuine attacks. The confusion matrix makes the consequence explicit: MNB produces 349 false negatives, nearly five times the 70 produced by SVC on the same 8,579 phishing test emails. In a deployed environment those false negatives represent malicious messages that pass through the model without a warning.

This behaviour is consistent with the modelling assumption described in Section 3.6.3. Multinomial Naive Bayes combines evidence under a conditional-independence assumption, effectively treating word features as independent once the class is known. That simplification is efficient, but phishing language often depends on combinations such as “verify your account immediately”, where the construction is more informative than isolated token probabilities. TF-IDF bigrams partially preserve local phrases, yet the classifier still combines their evidence under the Naive Bayes assumption. Figure 6 therefore supports SVC as the operationally stronger choice: it produces only 75 false positives and 70 false negatives, giving the most balanced confusion matrix and the lowest absolute error count among the evaluated models.

5.3 Operational Metrics Not Reported in the Base Paper

Method	False positives	False negatives	FP rate	Total errors
SVC / LinearSVC	75	70	0.0095	145
Random Forest	103	118	0.0130	221
Logistic Regression	114	118	0.0144	232
Multinomial Naive Bayes	78	349	0.0098	427
XGBoost	324	100	0.0409	424

Table 15: Error breakdown on the held-out test set (7,919 legitimate, 8,579 phishing). None of these figures are reported in [41].

Figure 4: Error-type breakdown — operational metrics not reported in the base paper [41]

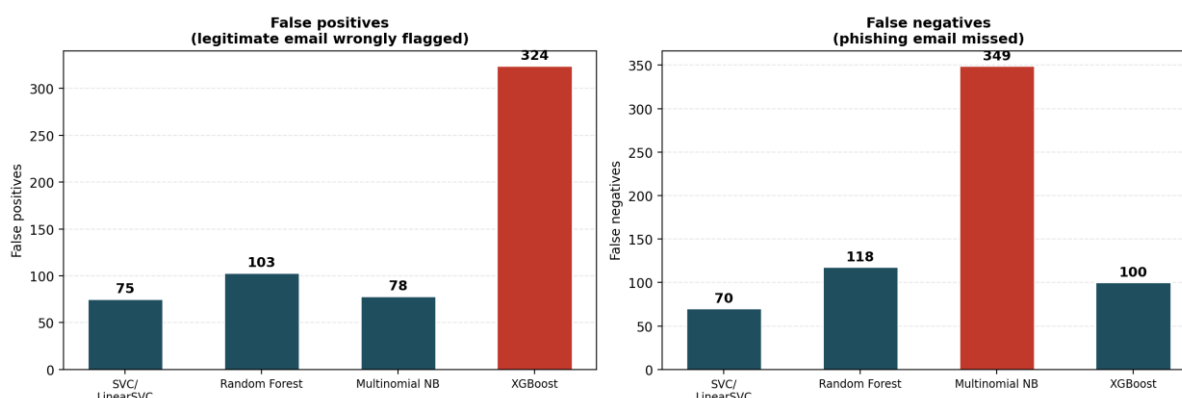


Figure 7: Error-type breakdown — operational metrics not reported in the base paper.

Headline F1 scores can conceal deployment behaviours that matter directly to users and administrators. Table 15 demonstrates this clearly. XGBoost and Multinomial Naive Bayes have almost identical total error counts—424 and 427 respectively—yet they fail in opposite directions. XGBoost generates 324 false positives and only 100 false

negatives, whereas Naive Bayes produces just 78 false positives but 349 false negatives. A summary score alone does not communicate this asymmetry.

For XGBoost, the dominant risk is over-flagging. Its false-positive rate is 4.09%, meaning legitimate messages are much more likely to be labelled as phishing than under the other models. In an operational system, repeated unnecessary warnings interrupt normal work, increase the cost of manual review and can produce alert fatigue. If users learn that many warnings are incorrect, they may begin ignoring or bypassing the system even when a genuine attack is detected. The model can therefore be statistically strong while still imposing an unacceptable trust cost.

Naive Bayes has the opposite problem. Its false-positive rate is only 0.98%, but 349 phishing emails are missed. Those false negatives are security exposures: the message reaches the user without the technical warning that the system is intended to provide. The model's high precision is therefore achieved partly by being conservative about when it raises an alert.

SVC's advantage is clearest when the same operational evidence is considered. It produces 75 false positives and 70 false negatives, only 145 errors in total, with a false-positive rate of 0.95%. It neither over-flags at XGBoost's level nor misses attacks at the Naive Bayes level. Reporting these absolute counts, rates and confusion matrices is a contribution relative to [41], which reports aggregate classification metrics but not this operational breakdown. The additional evidence supports model selection based on institutional consequences rather than headline F1 alone.

5.4 Extension Experiments and the Performance Ceiling

Experiment	F1 Result	Change from baseline
SVC baseline (replication)	99.16%	Reference
SVC + engineered features	99.16%	+0.00 — no change
Equal-weight voting ensemble	99.13%	−0.03 — slight degradation
Weighted voting ensemble (SVC ×3)	99.17%	+0.01 — negligible
SVC hyperparameter tuning (C = 2.0)	99.17%	+0.01 — negligible

Table 16: Results of four optimisation strategies applied beyond direct replication.

C value	F1 Score	Note
0.1	98.888%	Under-regularised for this data
0.3	99.097%	
0.5	99.115%	
1.0	99.155%	Default value used in replication
2.0	99.173%	Best observed
5.0	99.114%	
10.0	99.103%	Diminishing returns

Table 17: SVC regularisation parameter search. Full range spans 0.285 F1 points; the optimum sits 0.018 points above the default.

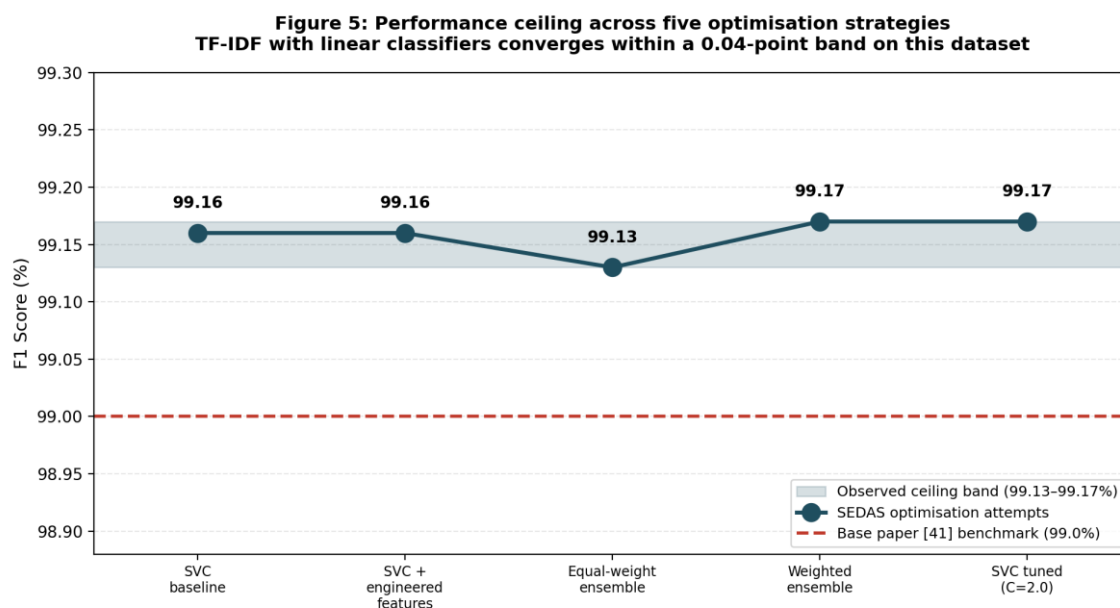


Figure 8: Performance ceiling across five optimisation strategies.

The four extension experiments produced the most substantive empirical result of the study because they tested whether a validated 99.16% SVC baseline could be meaningfully improved through several distinct strategies. None did. Engineered features remained at 99.16% F1, equal-weight voting decreased to 99.13%, and both the weighted ensemble and the best tuned SVC reached approximately 99.17%. The complete set therefore converges within a narrow 0.04-point band around the baseline.

The engineered-feature experiment tested the hypothesis that explicit phishing signals would add information not captured by TF-IDF. URL normalisation, HTML removal and features such as URL count, capital ratio, exclamation count and phishing-keyword count produced no improvement. A plausible explanation is redundancy. Terms such as “verify”, “urgent”, “password” and “account” already receive high TF-IDF weights when they discriminate between classes, while URL-like strings and punctuation patterns also influence the existing token representation. The added features therefore reorganised signals the model already possessed rather than introducing a new information source.

The equal-weight ensemble illustrates a different limitation. SVC was the strongest individual model, but assigning one vote each to SVC, Random Forest and Logistic Regression allowed the two weaker models to override SVC whenever they agreed. The resulting 99.13% F1 is slightly worse than SVC alone, showing that diversity is useful only when the additional models contribute complementary correct decisions. Majority voting does not automatically improve a strong base learner.

The 3:1:1 weighted ensemble then protected SVC from being outvoted. However, two one-vote models can never overturn three SVC votes, so the hard-voting rule mathematically collapses to the SVC decision on every example. Its 99.17% result consequently reflects the tuned SVC component rather than a genuine ensemble gain.

Hyperparameter tuning reached the same conclusion from another direction. Seven C values from 0.1 to 10.0 were tested. Performance improved from the strongly regularised end, peaked at C=2.0 with 99.173% F1, then declined slightly. The optimum is only 0.018 F1 points above the default C=1.0 result of 99.155%, and the full search spans only 0.285 points. The model is therefore already operating near a stable optimum for this representation.

A negative result obtained through controlled experiments is still a research finding. Here it has practical value because it narrows where additional effort should be invested. Further hand-crafted features, simple voting combinations or fine adjustment of C are unlikely to produce a substantive gain on this corpus. Meaningful improvement is more likely to require a different representation—such as contextual transformer embeddings, temporal information, richer URL/content fusion, or genuinely new data—rather than continued optimisation of the same saturated TF-IDF decision space.

5.5 Transformer-Based Comparison (Preliminary)

Parameter	Value	Rationale
Model	distilbert-base-uncased	Distilled BERT variant; CPU-feasible
Training sample	6,000 emails (7.3% of corpus)	CPU-only hardware constraint
Class balance	3,120 phishing / 2,880 legitimate	Stratified from the full corpus
Train / test split	4,800 / 1,200	80/20, stratified, random_state 42
Max sequence length	128 tokens	Balance of context and compute
Batch size / epochs	8 / 2	Memory and time constraints
Learning rate	2e-5	Standard for transformer fine-tuning

Table 18: DistilBERT preliminary experiment configuration.

Metric	Value	Note
Accuracy	97.92%	On 1,200 held-out emails
Precision	97.77%	Of emails flagged phishing, proportion correct
Recall	98.24%	Of actual phishing, proportion caught
F1 score	98.00%	Headline comparison figure
False-positive rate	2.43%	Higher than all classical methods except XGBoost
Confusion matrix	TN=562, FP=14, FN=11, TP=613	25 total errors from 1,200 test emails

Table 19: DistilBERT preliminary evaluation results.

Method	F1	Training set	Basis of comparison
SVC / LinearSVC	99.16%	65,988	Full dataset
Random Forest	98.71%	65,988	Full dataset
Logistic Regression	98.65%	65,988	Full dataset
DistilBERT (preliminary)	98.00%	4,800	7.3% subsample — not directly comparable
XGBoost	97.56%	65,988	Full dataset
Multinomial Naive Bayes	97.47%	65,988	Full dataset

Table 20: All six evaluated methods. The DistilBERT row is not directly comparable, having been trained and tested on a subsample.

Figure 7: All six evaluated methods against the published benchmark
Five classical methods (full dataset) and DistilBERT (preliminary, 6,000-email subsample)

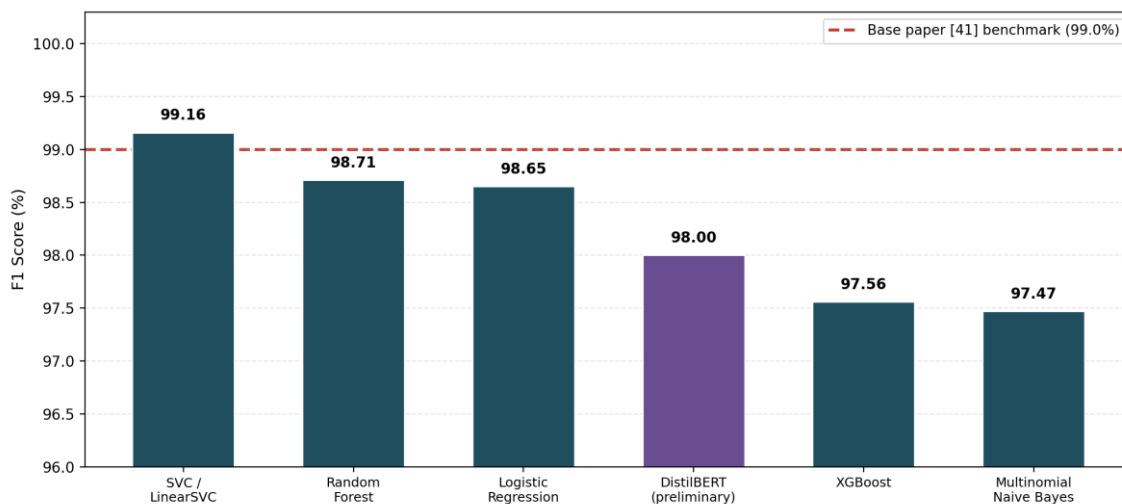


Figure 9: All six evaluated methods against the published benchmark.

Figure 8: Performance relative to training data volume
DistilBERT reached within 1.16 points of the best classical model using 7% of the training data

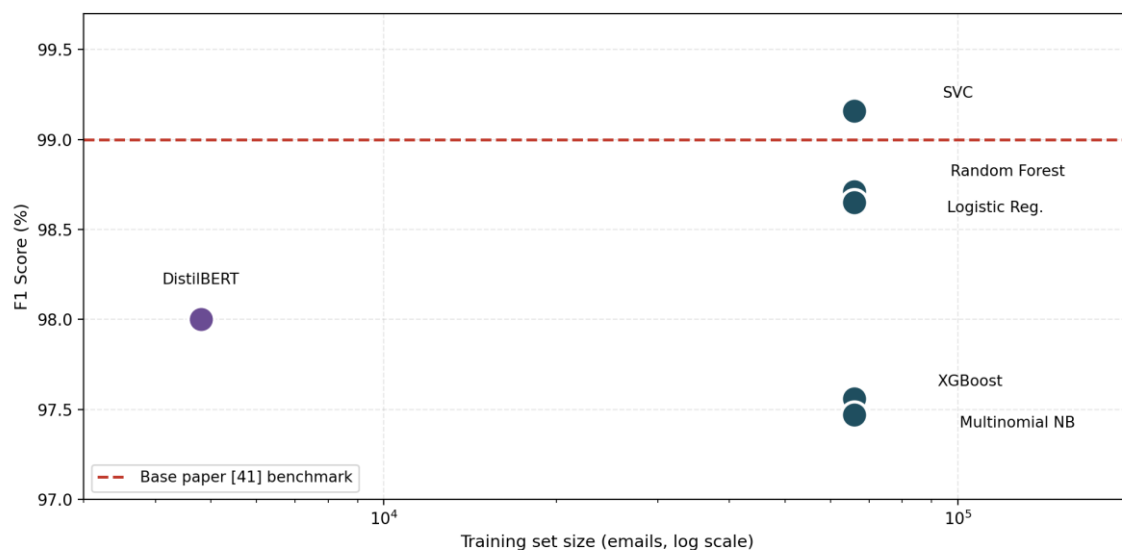


Figure 10: Performance relative to training data volume, logarithmic scale.

Figure 9: DistilBERT training loss
Epoch 1 converges steadily; epoch 2 loss rises slightly after batch 150, an early indication of overfitting on the reduced training sample

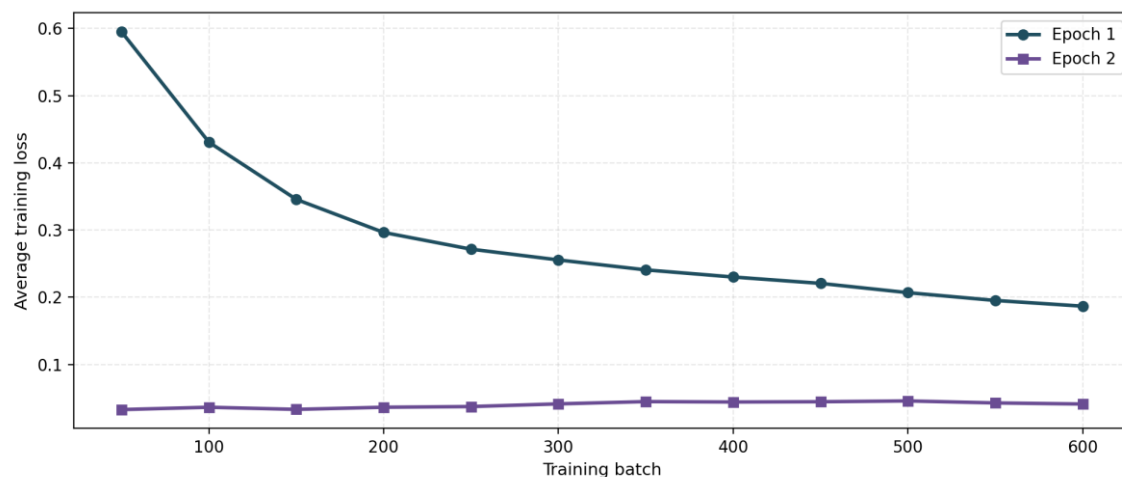


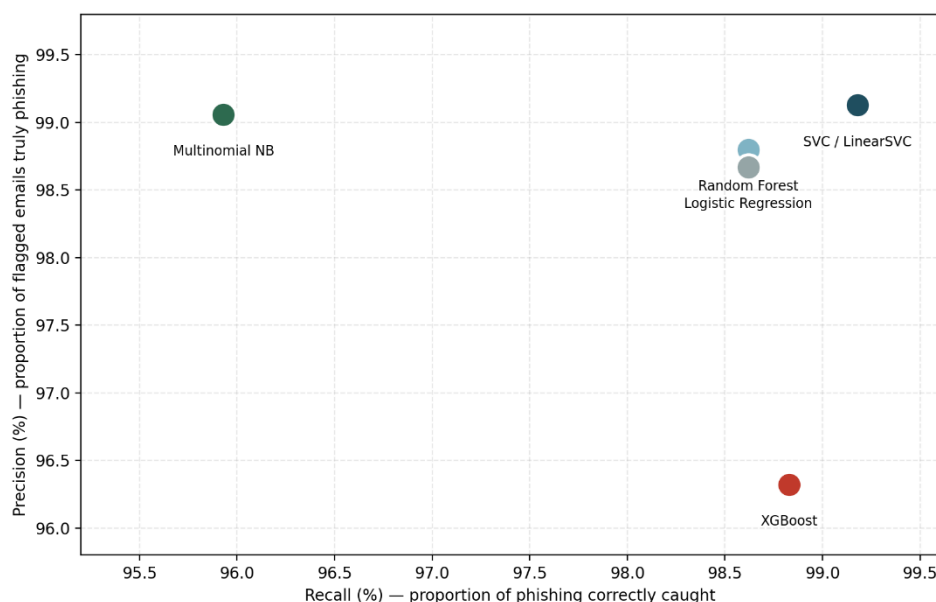
Figure 11: DistilBERT training loss across both epochs.

The DistilBERT experiment was designed to test whether contextual representation offers a viable alternative to the bag-of-words TF-IDF features used by all five classical models. Because the available hardware was CPU-only, the computational constraint was established before the result was interpreted. A stratified sample of 6,000 emails was used, containing 3,120 phishing and 2,880 legitimate messages. This produced 4,800 training examples and 1,200 held-out test examples. The model used a maximum sequence length of 128 tokens, batch size 8, two epochs and learning rate $2e-5$.

Under that preliminary configuration, DistilBERT achieved 97.92% accuracy, 97.77% precision, 98.24% recall and 98.00% F1. The held-out confusion matrix contains 562 true negatives, 14 false positives, 11 false negatives and 613 true positives, giving 25 errors from 1,200 examples. Its false-positive rate is 2.43%.

The comparison with the classical results must remain explicitly qualified. LinearSVC, Random Forest, Logistic Regression, XGBoost and Naive Bayes were trained on 65,988 emails and tested on 16,498. DistilBERT was trained on only 4,800 and tested on 1,200. The resulting figures are therefore indicative, not equivalent. It would be unsupported to conclude simply that DistilBERT “performed worse” than SVC because the models did not receive comparable data or compute.

**Figure 6: Precision-recall positioning of all five methods
Illustrating the operational trade-off each model makes**



The more informative observation is data efficiency. Figure 10 shows that DistilBERT reached within 1.16 F1 points of the full-data SVC while using roughly 7% of the corpus for training. This suggests that contextual representations are viable even under severe resource constraints. Training behaviour also shows why a larger experiment is necessary. Epoch 1 loss declined from 0.5951 to 0.1868, indicating rapid learning. Epoch 2 began at 0.0329 but later rose to approximately 0.0412, a pattern consistent with the onset of overfitting on the reduced sample. The 2.43% false-positive rate—higher than every classical method except XGBoost—is similarly consistent with a boundary learned from limited data.

The result neither confirms nor contradicts transformer advantages reported in [50] and [51], nor the more modest average margin reported in [49], because the sampling conditions differ. It establishes that the transformer approach is technically viable and sufficiently promising to justify full-corpus GPU evaluation, which is identified as a priority in Section 6.2.

5.6 Precision-Recall Trade-Off

Figure 12: Precision-recall positioning of all five classical methods.

Figure 12 shows that the five classical models occupy meaningfully different positions in precision-recall space. LinearSVC is the strongest because it does not obtain a high value on one axis by sacrificing the other: precision is 99.13% and recall 99.18%. This balance means that most alerts are correct while most genuine phishing is also detected. Its position therefore aligns with the low and symmetric false-positive/false-negative counts reported in Section 5.3.

Multinomial Naive Bayes is positioned toward high precision but lower recall. At 99.06% precision, its alerts are usually trustworthy, but 95.93% recall means it leaves substantially more phishing undetected. It behaves conservatively. XGBoost sits in the opposite operational direction: recall is high at 98.83%, but precision falls to 96.32% because the model flags many more legitimate messages. Random Forest and Logistic Regression cluster between these extremes, each providing comparatively balanced behaviour but at a lower overall level than SVC.

For an institution, the appropriate model is not determined by the highest recall or precision in isolation. A phishing defence must catch attacks without generating so many incorrect warnings that users lose confidence in the system. On the evaluated corpus, LinearSVC provides the most defensible balance: it minimises total errors, maintains a false-positive rate below 1%, and preserves recall above 99%. The precision-recall view therefore reinforces the deployment conclusion reached from the confusion matrices rather than introducing a conflicting ranking.

5.7 Significance of Findings

Reproducibility is the first significance of the study. The base paper [41] identifies limited dataset availability and incomplete methodological transparency as persistent problems in phishing research. SEDAS addresses that problem through an independent replication in which the dataset composition, feature configuration, train/test split, fixed seed and evaluation metrics are all disclosed. The 82,486-email corpus and its exact class counts match the benchmark, and the reproduced SVC result of 99.16% falls within the published 99.0% performance range. This does not prove superiority; instead, it provides evidence that the benchmark is reproducible by another team using an independently implemented pipeline. That result matters because later comparisons have a validated reference point. The repository, scripts and fixed seed make the work re-runnable rather than merely descriptive.

The second significance is the performance-ceiling finding. Four classical improvement strategies were tested systematically after the baseline was established, and none produced a meaningful gain. Engineered features added no improvement, equal voting slightly degraded performance, weighted voting reduced to the SVC decision, and tuning C moved F1 by only 0.018 points above the default. These are not four unrelated failures; together they provide convergent evidence that the current TF-IDF/classical configuration is saturated on this corpus. The preliminary DistilBERT experiment also did not exceed the numerical ceiling, although its reduced data volume prevents an equivalent comparison. The practical implication is that future researchers need not assume that another small lexical feature, simple hard-voting combination or minor SVC adjustment will automatically improve a 99% baseline. Substantive progress is more likely to require different representations, additional data or richer multimodal signals.

Third, the study shows why operational metrics are significant in security evaluation. F1 compresses precision and recall into one number, but it does not reveal how many legitimate messages are disrupted or how many attacks pass undetected. SEDAS therefore reports false-positive rates, absolute error counts and full confusion matrices. The contrast between XGBoost and Naive Bayes is especially instructive: they make almost the same total number of errors, yet XGBoost produces 324 false positives while Naive Bayes produces 349 false negatives. These behaviours impose very different institutional costs. By comparison, SVC produces only 145 total errors with 75 false positives and 70 false negatives. The richer evaluation therefore supports deployment reasoning that cannot be obtained from the base paper's aggregate figures alone.

Finally, the integration of the artefact is significant because social engineering is not purely a text-classification problem. A classifier can identify a suspicious email, but the attack still involves URLs, user judgement, repeated behaviour and organisational response. SEDAS places the validated classifier inside a wider system: URL verification combines first-seen heuristics with a tamper-evident registry; explainability turns a verdict into visible evidence; adaptive training converts user performance into targeted micro-lessons; and append-only attempts and registry records create governance data. The base paper's unit of contribution is primarily a deployed classifier, whereas SEDAS demonstrates how detection can feed into a layered defence process. That integration is particularly relevant to higher education, where diverse users and decentralised services make human judgement an unavoidable part of the security boundary.

Taken together, these four strands change the interpretation of the project. The evidence does not support a claim that SEDAS invented a universally better phishing classifier; rather, it shows where a strong published pipeline can be reproduced, where incremental classical optimisation stops yielding meaningful returns, how operational errors alter deployment choices, and how the classifier can be connected to controls that address the human and institutional dimensions of social engineering. This is practically useful because organisations do not deploy F1 scores in isolation. They deploy systems that must remain understandable, responsive, auditable and acceptable to users. The study therefore links experimental validity to operational decision-making and to a broader defence architecture, which is the significance required by the project problem rather than a claim based on a marginal numerical delta.

5.8 Limitations

Several limitations bound the conclusions of this study. First, the email corpus is assembled from public research collections rather than live traffic from the target higher-education environment. The dataset enables reproducible comparison with [41], but it does not establish how the models would behave under institutional domain shift, contemporary local terminology or a changing distribution of AI-assisted attacks. A production deployment would therefore require validation on representative institutional data under appropriate ethics and privacy controls.

Second, statistical significance testing against the published benchmark was not possible. Reference [41] reports summary performance figures but does not provide fold-level scores, repeated-run variance or prediction outputs from which confidence intervals for the difference could be reconstructed. Small numerical deltas between the two studies should consequently be treated as descriptive rather than statistically significant.

Third, the extension methods were reasonable but not exhaustively optimised. XGBoost, Random Forest and ensemble configurations could be tuned over much larger search spaces, so the study establishes the performance of the tested settings rather than the theoretical maximum of those algorithm families. The same qualification applies more strongly to DistilBERT. It was trained on 4,800 emails and tested on 1,200, compared with 65,988/16,498 for the classical models. Its 98.00% F1 is therefore not directly comparable and cannot establish whether the observed classical ceiling also applies to transformers at full scale.

Fourth, the awareness component was validated technically but not behaviourally. No real user population was available to measure whether adaptive training reduces risky clicks over time, so the Part A behavioural KPI remains unmeasured. Finally, authentication and role-based access control were not implemented at MVP scope. This leaves report submission and risk-changing endpoints unsuitable for open production exposure. These limitations are explicitly identified so that the claims remain bounded to what the artefact and experiments actually demonstrate.

5.9 Outcomes Against Objectives

Objective (Part A)	Target	Achieved	Status
Detection F1	≥ 0.95	0.9917 (tuned SVC)	Exceeded

Objective (Part A)	Target	Achieved	Status
False-positive rate	< 2%	0.95%	Exceeded
Classification latency	< 2,000 ms	≈ 7-11 ms	Exceeded
URL verification latency	< 2,000 ms	≈ 1.4-9.3 ms	Exceeded
Registry tamper-evidence	Verifiable chain	Chain validation confirmed	Met
Adaptive training	Persona-based delivery	Implemented and verified	Met
Risky-click reduction	30% (simulated)	Not measured	Deferred

Table 21: Outcomes against the objectives defined in Part A.

Table 21 shows that the project met or exceeded most of the measurable objectives established in Part A. Detection quality exceeded the minimum F1 target of 0.95, reaching 0.9917 with the tuned SVC. The false-positive objective was also exceeded: 0.95% is less than half of the 2% threshold. Both latency objectives were surpassed by large margins. Classification required approximately 7-11 ms against a 2,000 ms target, while URL verification required about 1.4-9.3 ms. Registry tamper-evidence was met through full-chain validation, and adaptive training was implemented and verified through the persona, risk-band and lesson-selection tests. These results mean that six technical objectives were either met or exceeded, with the quantitative detection and latency targets comfortably surpassed rather than narrowly satisfied.

The project also produced outcomes that were not explicit in the original objective table. It independently replicated the Q1 benchmark on the identical corpus, evaluated six classification methods, reported operational error behaviour absent from the base paper, and produced a controlled performance-ceiling finding from four optimisation experiments. These outcomes strengthen the evidence base beyond simple requirement completion.

The one objective not measured is the 30% reduction in risky clicks. Demonstrating that outcome requires a real or ethically approved simulated phishing campaign with human participants, baseline and follow-up behaviour, sufficient observation time and appropriate consent/governance. Such a study was outside the implementation scope and would require institutional ethics approval. The KPI is therefore deferred rather than claimed as achieved.

5.10 Design Trade-Offs

Several design choices in SEDAS deliberately exchange production-level capability for evidence, accessibility or implementation feasibility. The URL trust layer illustrates the first trade-off. A SHA-256 hash chain was selected instead of the Part A distributed ledger. Tamper-evidence is retained because any historical modification breaks the cryptographic sequence, but decentralised consensus is given up. This removes gas fees, wallet management and blockchain latency, which is appropriate for an MVP, while leaving distributed anchoring as future work.

The modelling strategy also balances classical and transformer approaches. TF-IDF with LinearSVC is fast, sparse, computationally inexpensive and straightforward to explain through feature coefficients. What it gives up is the richer context and word-order representation available to transformers. DistilBERT was therefore evaluated as an extension rather than used to replace the operational baseline before an equivalent full-data comparison existed.

Microservices were retained instead of collapsing the platform into a monolith. The benefit is independent development, scaling, testing and failure isolation for detection, URL verification and training. The cost is operational overhead: three processes, separate datastores and more network boundaries must be managed, without the gateway or service discovery that a production microservice deployment would normally include. At MVP scale, the clear separation of responsibilities justified that cost.

Hardware introduced another explicit trade-off. CPU-only execution improves accessibility and reproducibility because the artefact runs on ordinary development machines, but it prevents practical full-corpus transformer fine-tuning. The reduced DistilBERT experiment is the direct consequence. A GPU would increase experimental scope but reduce the “run anywhere” property of the current build.

Finally, CORS was configured permissively for local browser demonstration. This simplifies integration during development but would be unsafe for production because arbitrary origins could invoke the services. Restricting allowed origins, adding institutional authentication and enforcing rate limits are therefore deployment requirements. In each case, the MVP choice preserves the behaviour needed for assessment while making the sacrificed production property explicit rather than implicit.

6. Conclusion and Future Work

6.1 Conclusion

This project investigated phishing defence as both a machine-learning problem and a broader social-engineering problem. The starting point was the reproducibility of a strong published benchmark: Al-Subaiey et al. [41] reported approximately 99% F1 for an SVM-based phishing detector on a large public corpus. SEDAS independently reproduced that task on the identical 82,486-email dataset, then extended the work through additional models, controlled optimisation experiments, richer operational metrics and an integrated defensive architecture.

The replication was successful. LinearSVC achieved 99.16% F1, closely confirming the published 99.0% result, while Random Forest and Multinomial Naive Bayes remained in the same general performance range as [41]. Logistic Regression and XGBoost did not exceed the benchmark. More importantly, four targeted optimisation strategies also failed to produce a meaningful gain: engineered features remained at 99.16%, equal voting fell slightly, and weighted voting and SVC tuning reached only about 99.17%. The convergence of these results supports the main empirical finding that the evaluated TF-IDF/classical pipeline is operating near a practical performance ceiling on this corpus.

Operational analysis strengthened that conclusion. SVC produced the lowest total error count, with 75 false positives and 70 false negatives on 16,498 held-out emails. By contrast, XGBoost over-flagged legitimate messages and Naive Bayes missed substantially more attacks. Reporting these counts, false-positive rates, confusion matrices and measured latency adds deployment evidence not available from headline F1 alone. The preliminary DistilBERT experiment demonstrated that contextual modelling is viable, reaching 98.00% F1 from only 4,800 training emails, but the reduced sample prevents a definitive comparison with the full-data classical models.

The project's guiding questions can therefore be answered directly. First, the published benchmark can be independently reproduced on the identical public data, with SVC again emerging as the strongest replicated method. Second, the tested classical improvements—feature engineering, simple ensembling and regularisation tuning—do not materially exceed that benchmark. Third, a transformer approach is technically viable and data-efficient at preliminary scale, but a GPU-enabled full-corpus experiment is required before any superiority claim can be made. Fourth, phishing detection can be integrated meaningfully with human-layer defence: SEDAS combines explainable classification, heuristic and registry-based URL verification, adaptive training, tamper-evident records and governance-oriented data within one working system.

The contribution is therefore not a marginal score increase. It is a reproducible validation of a Q1 benchmark, controlled evidence about the limit of further classical optimisation, operational error analysis that changes how model suitability is interpreted, and an implemented architecture that connects technical detection to the user and governance layers social engineering actually exploits.

6.2 Future Work

Future work should first convert the preliminary transformer experiment into an equivalent full-scale comparison. DistilBERT should be fine-tuned on the complete training corpus using GPU resources, with the same held-out evaluation protocol used by the classical models. Larger transformer variants such as BERT and RoBERTa should then be evaluated because [50] and [51] report advantages for contextual models on related phishing corpora. This would determine whether the classical performance ceiling identified here is a property of the TF-IDF representation rather than of the dataset itself.

The classical extension space also remains open. XGBoost was evaluated with a reasonable fixed configuration, but a structured hyperparameter search could test whether its weak precision on sparse features can be improved. Alternative boosting formulations or dimensionality reduction could be examined before concluding that tree boosting is unsuitable for this representation.

Production security is a second priority. FR-08 should be completed through institutional single sign-on and role-based access control, ensuring that only authorised identities can submit registry reports, alter training state or

view governance data. The URL report endpoint should add rate limiting and authenticated attribution to reduce poisoning risk, and CORS should be restricted to approved origins.

A real institutional pilot would enable evaluation of the project's deferred behavioural objective. With ethics approval, simulated campaigns could measure baseline risky-click behaviour, training exposure, follow-up performance and retention over time. The awareness content library should also be expanded across additional roles and attack patterns so adaptation is based on a richer set of scenarios. Particular attention should be given to AI-generated phishing, which recent work [43], [57] identifies as an emerging threat capable of producing more fluent and personalised social-engineering content. Together, these extensions would move SEDAS from a validated MVP toward a production-ready and behaviourally evaluated institutional defence platform.

References

APA 7th edition, continuing the Part A reference register [1]-[40].

Base paper

[41] Al-Subaiey, A., Al-Thani, M., Alam, N. A., Antora, K. F., Khandakar, A., & Zaman, S. M. A. U. (2024). Novel interpretable and robust web-based AI platform for phishing email detection. *Computers and Electrical Engineering*, 120, 109625. <https://doi.org/10.1016/j.compeleceng.2024.109625> [Q1]

Additional references added in Part B

[42] Champa, A. I., Rabbi, M. F., & Zibran, M. F. (2025). Improving phishing email detection performance through deep learning with adaptive optimization. *Scientific Reports*, 15, 36724. [Q1]

[43] Opara, C., Modesti, P., & Golightly, L. (2025). Evaluating spam filters and stylometric detection of AI-generated phishing emails. *Expert Systems with Applications*, 276, 127044. [Q1]

[44] Alotaibi, S. R., et al. (2025). Explainable artificial intelligence in web phishing classification on secure IoT with cloud-based cyber-physical systems. *Alexandria Engineering Journal*, 110, 490-505. [Q1]

[45] (2026). Designing effective phishing awareness training: The role of feedback and engagement strategies. *International Journal of Human-Computer Studies*. [Q1]

[46] (2026). An explainable transformer-based model for phishing email detection: A large language model approach. [Q1]

[47] Uddin, M. A., Islam, M. N., Maglaras, L., Janicke, H., & Sarker, I. H. (2025). ExplainableDetector: Exploring transformer-based language modeling approach for SMS spam detection with explainability analysis. *Digital Communications and Networks*, 11(5), 1504-1518. [Q1]

[48] Sarker, I. H., Janicke, H., Mohsin, A., Gill, A., & Maglaras, L. (2024). Explainable AI for cybersecurity automation, intelligence and trustworthiness in digital twin. *ICT Express*, 10(4), 935-958. [Q1]

[49] Alhuzali, A., Alloqmani, A., Aljabri, M., & Alharbi, F. (2025). In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets. [Verify]

[50] (2026). Phishing Email Detection Using BERT and RoBERTa. [Q2]

[51] (2025). Comparative Investigation of Traditional Machine Learning Models and Transformer Models for Phishing Email Detection. [Preprint]

[52] (2026). A Temporally Dynamic Feature-Extraction Framework for Phishing Detection with LIME and SHAP Explanations. *Future Internet*. [Q2]

[53] Uddin, M. A., et al. (2025). Explainable Machine Learning for Phishing Site Detection: A High-Efficiency Approach Using Boosting Models and SHAP. *The Journal of Engineering*. [Q2]

[54] (2026). Hybridizing Explainable AI (XAI) for Intelligent Feature Extraction in Phishing Website Detection. *Electronics*. [Q2]

[55] MLPhishChain: A machine learning-based blockchain framework for reducing phishing threats. *Frontiers in Blockchain*. [Q2]

[56] Alluqmani, K., et al. (2025). Assessing the Efficacy of Security Awareness Training in Mitigating Phishing Attacks: A Review. *International Journal of Advanced Trends in Computer Science and Engineering*, 14(3), 177-184. [Verify]

[57] (2025). Machine Learning and Watermarking for Accurate Detection of AI-Generated Phishing Emails. *Electronics*, 14(13), 2611. [Q2]

Appendices

- Appendix A — Complete results data (Tables A1 and A2 below)
- Appendix B — Technical and Programmer Manual (separate document)
- Appendix C — User Guide (separate document)
- Appendix D — Consolidated Project Log, Weeks 1-7 (separate document)
- Appendix E — Meeting Minutes #1 to #5
- Appendix F — Evidence screenshots (docs/evidence/)
- Appendix G — Source code repository

Appendix A: Complete Results Data

Method	Accuracy	Precision	Recall	F1	FP rate
SVC / LinearSVC	99.12%	99.13%	99.18%	99.16%	0.0095
Random Forest	98.66%	98.80%	98.62%	98.71%	0.0130
Logistic Regression	98.59%	98.67%	98.62%	98.65%	0.0144
XGBoost	97.43%	96.32%	98.83%	97.56%	0.0409
Multinomial Naive Bayes	97.41%	99.06%	95.93%	97.47%	0.0098
DistilBERT (preliminary)	97.92%	97.77%	98.24%	98.00%	0.0243

Table A1: Complete evaluation metrics. The DistilBERT row derives from a 1,200-email subsample test set; all others from the 16,498-email held-out set.

Method	TN	FP	FN	TP
SVC / LinearSVC	7,844	75	70	8,509
Random Forest	7,816	103	118	8,461
Logistic Regression	7,805	114	118	8,461
XGBoost	7,595	324	100	8,479
Multinomial Naive Bayes	7,841	78	349	8,230
DistilBERT (preliminary)	562	14	11	613

Table A2: Confusion matrix values. TN = true negative, FP = false positive, FN = false negative, TP = true positive.